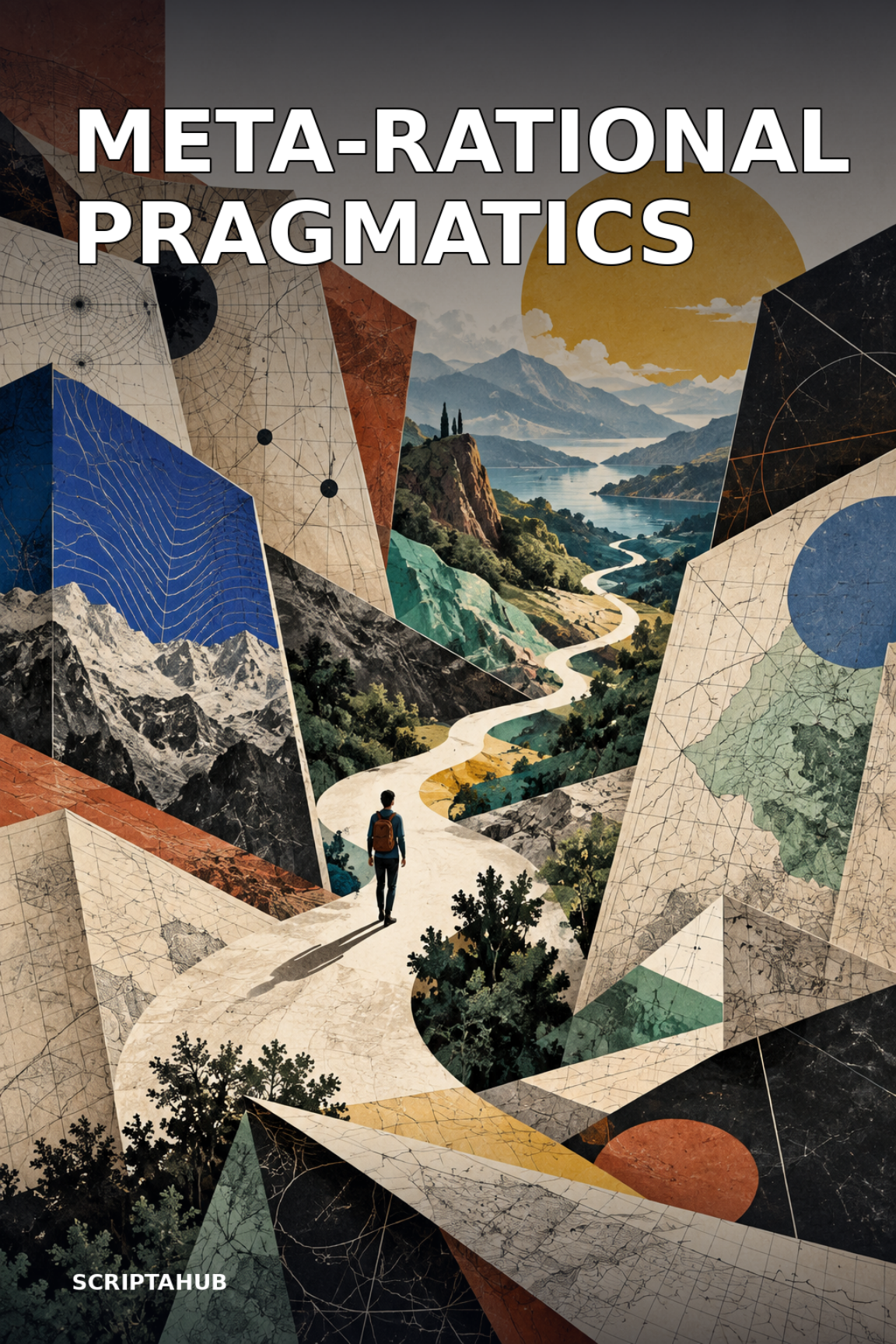
META-RATIONAL PRAGMATICS



SCRIPTAHUB

PROGRAM DE CERCETARE

META-RATIONAL PRAGMATICS

Înțelegerea ca învățare a
distincțiilor executabile

Fundamente, arhitectură și aplicații

Sînică Alboaie

Concepție de cercetare și direcție intelectuală
Elaborare asistată de AI

CURS • MODEL OPERAȚIONAL • EXPERIMENTE PROPUSE

Ediție de studiu | Septembrie 2026

Copyright

© 2026 Axiologic Research SRL. Toate drepturile rezervate.

Axiologic Research SRL a creat și a contribuit cu versiunea inițială a acestei cărți în cadrul cazului de utilizare SCRIPTA din proiectul Achilles.

Ediția 1 · publicată pe 2026-09-11 pe ScriptaHub (scriptahub.com).

Titlul în engleză: Meta-Rational Pragmatics.

Cuprins

Prefață. O ipoteză care trebuie să poată pierde 4

I / Întrebarea și limbajul

1. Înțelegerea ca problemă inginerească 6

2. Vocabularul matematic minimal 8

3. Programe, semantică, interpretare 11

4. Probabilitate, decizie, adevăr 13

II / Fundamente pentru distincții

5. Reprezentări și echivalențe 15

6. Învățarea stărilor 17

7. Predicție și intervenție 19

8. Interpretări abstracte 21

III / Un mecanism de învățare

9. Diagnosticul erorilor 23

10. Inducția interpretoarelor 26

11. Raționalitatea calculului 28

12. Memorie și transfer 30

Cuprins · continuare

IV / Formele cunoașterii

13. Distanțe pragmatice 32

14. Conuri și coordonate 34

15. Un atlas de interpretoare 37

V / De la model la sistem

16. Arhitectura unei tehnologii MRP 39

17. Execuție, dovezi și revizuire 42

18. MRP-VM și SOP Lang 45

VI / Aplicații și validare

19. Verificare software și protocoale 47

20. Documente, sinteză și proceduri 49

21. Cercetare și descoperire 51

22. Educație și agenți interactivi 53

23. Programul experimental 55

24. Limite, obiecții și contribuții 58

Anexe / Instrumente pentru studiu și verificare

A. Rezultate matematice elementare 61

B. Exerciții cu soluții comentate 64

C. Glosar operațional 66

D. Bibliografie comentată 68

PREFAȚĂ

O ipoteză care trebuie să poată pierde

O tehnologie a înțelegerii nu are nevoie, pentru a începe, de o definiție definitivă a minții. Are nevoie de situații în care un agent confundă lucruri pe care ar trebui să le distingă, de operații prin care poate corecta confuzia și de criterii independente pentru a verifica dacă a progresat. Această carte dezvoltă Meta-Rational Pragmatics, prescurtat MRP, pornind de la această cerință.

Intuiția centrală este că înțelegerea unui agent nu se află exclusiv într-un depozit de reprezentări. Ea depinde și de procedurile prin care agentul construiește reprezentările, le folosește, le compară cu experiența și le înlocuiește. Unele proceduri interpretează un text, altele urmăresc o succesiune de evenimente, estimează un rezultat, verifică o demonstrație sau produc o procedură nouă. În acest sens, agentul acumulează nu numai răspunsuri, ci și moduri de a face anumite diferențe vizibile.

Cartea transformă această intuiție într-o propunere inginerească: o familie adaptivă de interpretoare, controlată prin teste, contraexemple, costuri și evidențe. Expresia „distincție executabilă” numește o diferență pe care un program o poate calcula și a cărei relevanță poate fi verificată. De exemplu, ordinea dintre acordarea și revocarea unei permisiuni este o distincție executabilă. Tonul convingător al unei explicații nu este, singur, o dovadă că agentul a păstrat-o.

Lucrarea este organizată ca un curs progresiv. Introduce obiectele matematice înainte de a le folosi; explică relațiile de echivalență înaintea automatului învățat; separă probabilitatea de utilitate înaintea metaraționamentului; introduce distanțele și convexitatea înaintea conurilor. Cititorul nu

trebuie să cunoască interpretarea abstractă, reprezentările predictive sau sinteza de programe. Familiaritatea elementară cu programele și tabelele este utilă, dar exemplele nu cer citirea unui anumit limbaj de programare.

MRP nu este prezentată ca o teorie validată a cogniției umane și nici ca un produs deja realizat. Elementele consacrate sunt atribuite literaturii. Rezultatele matematice elementare sunt demonstrate în limitele ipotezelor lor. Componentele arhitecturale reprezintă propuneri. Exemplele numerice sunt calcule didactice, nu măsurători ale unui prototip. Aplicațiile sunt analizate ca oportunități de testare, nu ca promisiuni comerciale îndeplinite.

Statut	Ce înseamnă în această carte	Ce nu autorizează
Fundament consacrat	O idee sau metodă documentată în literatura citată.	Transferul automat al tuturor garanțiilor sale la MRP.
Rezultat dedus	O concluzie demonstrată sub ipoteze explicite.	Generalizarea dincolo de acele ipoteze.
Mecanism propus	O regulă de proiectare suficient de precisă pentru implementare.	Afirmația că implementarea există sau este competitivă.
Ipoteză empirică	O predicție evaluabilă prin comparație și experiment.	Tratarea unei intuiții coerente drept rezultat observat.

Citarea folosește identificatori de forma [Cousot1977]. Bibliografia oferă lucrările și rolul lor în argument. Ea este selectivă: susține mecanismele discutate, fără a pretinde o inventariere exhaustivă a tuturor cercetărilor despre agenți sau înțelegere. Numele MRP desemnează programul formulat aici; folosirea lui nu constituie o revendicare de prioritate asupra metaraționamentului, pragmatismului sau învățării de programe.

Concepția de cercetare îi aparține lui Sînică Alboaie. Elaborarea conceptuală, expunerea și verificarea bibliografică au fost asistate de AI. Textul nu a parcurs o evaluare științifică independentă. Cititorul este invitat să păstreze mecanismele care rezistă criticii și să respingă formulările care nu produc diferențe măsurabile.

PARTEA I / Întrebarea și limbajul

CAPITOLUL 01

Înțelegerea ca problemă inginerească

De la impresie la capacitate

Să presupunem că un asistent primește două înregistrări: „acces acordat, apoi revocat” și „acces revocat, apoi acordat”. El identifică aceleași persoane, aceleași resurse și aceleași două operații în ambele înregistrări. Totuși, dacă regu-

la sistemului este că ultima operație prevalează, cele două situații nu permit aceeași decizie. Un vocabular bogat despre acces nu compensează pierderea ordinii.

Acest exemplu fixează o întrebare mai îngustă decât „înțelege agentul cu adevărat?”. Întrebarea este dacă agentul dispune de reprezentarea și procedura necesare pentru a răspunde corect unei familii de întrebări. Vom numi această proprietate înțelegere operațională. Termenul nu rezolvă disputa despre conștiință, intenționalitate sau experiență subiectivă; selectează o dimensiune care poate fi investigată prin sisteme construite și comparate.

Diferența dintre cele două întrebări contează metodologic. Pentru prima, comportamente identice pot primi interpretări filosofice diferite. Pentru a doua, putem fabrica exemple cu răspunsuri cunoscute, modifica informația disponibilă și observa care componentă permite progresul. O asemenea restrângere nu face problema trivială. Capacitatea de a descoperi ce trebuie reprezentat, fără ca proiectantul să furnizeze fiecare distincție, este tocmai problema dificilă.

Trei teze diferite

O primă teză spune că orice comportament inteligent poate fi descris printr-o familie de interpretoare. Este plauzibilă ca perspectivă de organizare, dar prea permisivă pentru a funcționa singură ca ipoteză empirică. Orice program suficient de complex poate fi împărțit retrospectiv în module și numit astfel.

A doua teză spune că anumite erori apar deoarece reprezentarea disponibilă identifică situații care cer rezultate diferite. Această teză este precisă și poate primi demonstrații locale. Dacă două intrări sunt transformate exact în aceeași va-

loare, iar un mecanism ulterior vede numai acea valoare, mecanismul nu poate răspunde diferit în funcție de informația deja pierdută.

A treia teză este pariul MRP: în anumite familii de probleme, un agent care detectează asemenea coliziuni și învață interpretoare reutilizabile va obține un compromis mai bun între eroare, cost și adaptabilitate decât alternativele comparabile. Aceasta nu urmează logic din primele două. Necesită o arhitectură concretă și experimente care includ toate costurile.

Cele trei niveluri nu trebuie amestecate. O demonstrație despre informația pierdută nu demonstrează superioritatea unui produs. Un prototip util într-o microlume nu demonstrează o teorie universală a minții. Invers, faptul că o afirmație filosofică rămâne discutabilă nu împiedică testarea mecanismului ingineresc.

Reprezentare și interpretare

O reprezentare este o structură pe care agentul o poate stoca sau manipula: un număr, un tabel, un vector, un graf, o expresie ori un program. Un interpretor este o procedură care atribuie unor astfel de structuri consecințe operaționale: răspunsuri, transformări, predicții sau acțiuni. Cele două sunt legate, dar nu identice.

Un tabel cu evenimente nu produce singur starea curentă. Este necesară o regulă pentru ordonare și actualizare. Aceeași regulă, aplicată unui tabel în care lipsesc momentele sau indicii de ordine, poate deveni inutilizabilă. Capacitatea aparține ansamblului reprezentare–operație, nu unei jumătăți izolate a acestuia.

Problema ancorării simbolurilor, formulată de Harnad, întreabă cum pot simbolurile să fie legate de ceva dincolo de alte simboluri [Harnad1990]. MRP nu pretinde că închide această problemă filosofică. O tratează practic prin legături verificabile între reprezentări, surse, senzori și efectele acțiunilor. Un identificator dintr-un raport devine operațional când poate fi urmărit până la observația sau obiectul la care se referă.

Această ancorare poate fi indirectă. Un agent care analizează documente nu trebuie să posede senzori proprii pentru fiecare afirmație. Dar trebuie să distingă „am observat”, „sursa afirmă”, „rezultă sub ipotezele modelului” și „propun ca posibilitate”. Distincțiile sunt parte din arhitectură, nu simple formule de politețe.

Pragmatic nu înseamnă oportunist

În această carte, pragmatica privește folosirea unei interpretări într-un context, pentru un scop și sub constrângeri. Ea nu înseamnă că o propoziție devine adevărată dacă este convenabilă. Un sistem care învață să producă aprobarea utilizatorului prin răspunsuri false poate avea succes pe o recompensă prost definită și poate eșua epistemic.

Din acest motiv, proiectarea trebuie să separe trei evaluări. Corectitudinea verifică dacă rezultatul respectă regulile asumate. Adecvarea empirică verifică dacă predicțiile se potrivesc observațiilor. Utilitatea verifică dacă decizia servește scopul legitim al sarcinii. Un scor unic poate fi util într-un experiment bine delimitat, dar nu trebuie să ascundă diferențele dintre aceste evaluări.

„Meta-rațional” numește alegerea și dezvoltarea modului de raționare. Un agent poate decide că o comparație numerică este suficientă, că trebuie urmărit timpul sau că lipsește o

observație. Nu este o logică magică deasupra tuturor logicilor. Este administrarea unor resurse cognitive și computaționale finite.

O definiție de lucru

MRP va desemna un mecanism prin care un agent construiește, selectează, compune și revizuiește interpretoare, folosind consecințe verificabile și costuri declarate. Un progres MRP apare când această reorganizare produce competență pe cazuri noi, nu doar corectarea cazului care a declanșat-o.

Definiția lasă deschise implementările. Interpretoarele pot fi simbolice, numerice, neuronale sau hibride. Unele pot fi furnizate de proiectant; altele pot fi învățate. Pentru a evita confuzia, fiecare experiment va raporta separat ce a fost oferit, ce a fost indus și ce a fost doar selectat. Această contabilitate este prima condiție a unui rezultat defensabil.

CAPITOLUL 02

Vocabularul matematic minimal

Mulțimi, funcții, relații

O mulțime este o colecție de obiecte tratate ca elemente distincte. În exemplul nostru, mulțimea evenimentelor posibile poate conține acordarea și revocarea accesului. O secvență

diferă de o mulțime: păstrează ordinea și, de regulă, repetările. Secvența „acordare, revocare” nu este secvența „revocare, acordare”, deși mulțimea elementelor lor este aceeași.

O funcție asociază fiecărei intrări admise un singur rezultat. Scrierea R(h) înseamnă rezultatul funcției de reprezentare R aplicate istoricului h. Dacă R păstrează numai mulțimea evenimentelor, cele două istorii au aceeași imagine. O funcție parțială nu este definită pentru toate intrările; de exemplu, „ultimul eveniment” nu există pentru o secvență goală dacă nu introducem o convenție suplimentară.

O relație nu trebuie să ofere un rezultat unic. Relația „depinde de” poate conecta un rezultat la mai multe surse. Relația „este compatibil cu” poate conecta o observație la mai multe modele. Distincția dintre funcție și relație ne permite să păstrăm ambiguități fără a le transforma prematur într-o alegere arbitrară.

O compoziție execută o transformare după alta. Dacă E extrage evenimentele și R le reduce la o stare, atunci R(E(text)) desemnează întregul traseu. Compoziția este validă numai dacă rezultatul primei operații este o intrare admisă de a doua. Această condiție, simplă matematic, va deveni un contract important între interpretoare.

Echivalență și partiție

O relație de echivalență este reflexivă, simetrică și tranzitivă. Reflexivitatea cere ca orice obiect să fie echivalent cu el însuși. Simetria cere ca echivalența să funcționeze în ambele direcții. Tranzitivitatea cere ca, dacă primul obiect este echivalent cu al doilea și al doilea cu al treilea, primul să fie echivalent cu al treilea.

O asemenea relație împarte domeniul în clase disjuncte, numite clase de echivalență. Colecția claselor este o partiție. Dacă grupăm istorii după rezultatul R(h), obținem automat o partiție: aceeași valoare R înseamnă aceeași clasă. O reprezentare poate fi înțeleasă, așadar, drept o modalitate de a spune ce diferențe ignorăm.

O partiție rafinează alta atunci când separă unele clase fără să unească elemente care erau deja separate. De exemplu, clasificarea documentelor după tip și an rafinează clasificarea numai după tip. Ea poate păstra informația veche și adăuga informație nouă. Nu rezultă însă că va produce automat un algoritm mai bun: poate crește costul și poate cere mai multe date pentru estimare.

Vom folosi uneori termenul „spațiu cât". El desemnează mulțimea claselor de echivalență, nu un spațiu geometric special. Înlocuim fiecare istoric concret cu clasa tuturor istoriilor considerate echivalente. Geometria, dacă există, trebuie adăugată ulterior printr-o distanță sau altă structură.

Ordine și grafuri

O ordine parțială permite compararea unor perechi, fără să oblige la compararea tuturor. Un interpretor care păstrează ordinea evenimentelor și unul care păstrează sursele documentare pot fi incomparabili: fiecare conservă o informație pe care celălalt o ignoră. Nu există neapărat un singur șir de reprezentări, de la „proastă" la „bună".

Un graf este format din noduri și legături. În MRP, nodurile pot fi operații sau rezultate; legăturile pot reprezenta dependențe. Dacă un calcul trebuie să primească mai întâi două rezultate, graful precizează această condiție. Un graf orientat fără cicluri permite ordonarea operațiilor astfel încât fiecare să vină după dependențele sale. Aceasta se numește ordonare topologică.

Un ciclu nu este obligatoriu o eroare. Poate descrie repetarea unei actualizări până la stabilizare. Dar trebuie introdusă o regulă care explică ce se repetă, ce stare se schimbă și când se oprește procesul. Un desen circular nu este încă un algoritm executabil.

Predicat și invariant

Un predicat este o proprietate care poate fi adevărată sau falsă despre un obiect, în modelul ales. „Toate evenimentele au un indice de ordine” este un predicat asupra unui tabel. Un invariant este o proprietate care trebuie să rămână adevărată pe parcursul unei execuții sau al unei clase de transformări.

Pentru o bibliotecă MRP, un invariant posibil este că orice rezultat acceptat păstrează identificatorii surselor folosite. Nu înseamnă că sursele sunt adevărate; înseamnă că originea rezultatului nu se pierde. Alt invariant poate cere ca o operație declarată numai pentru citire să nu modifice starea externă.

O demonstrație pornește din ipoteze și folosește reguli explicite pentru a obține o concluzie. Un test execută unul sau mai multe cazuri. O demonstrație poate acoperi toate cazurile din model; un test nu primește această putere doar pentru că este automatizat. La rândul ei, o demonstrație nu verifică singură dacă modelul descrie corect lumea reală.

Cost și dimensiune

Dimensiunea unei probleme poate fi numărul evenimentelor, numărul interpretoarelor candidate sau lungimea maximă a programelor căutate. O procedură liniară vizitează un număr de elemente proporțional cu dimensiunea. Una expo-

nențială poate dubla spațiul de căutare la fiecare extensie cu o unitate. Aceste diferențe explică de ce o idee logic corectă poate fi impracticabilă.

Noțiunile de calculabilitate și complexitate separă existența unui algoritm de resursele necesare executării lui. Tratările introductive din informatica teoretică dezvoltă această distincție sistematic [Barak]. Pentru MRP, ea se traduce într-o cerință: fiecare mecanism de inducție trebuie să aibă limite de căutare, de timp și de memorie, chiar dacă descrierea sa conceptuală este foarte generală.

Obiect matematic	**Exemplu MRP**	**Întrebarea practică**
Funcție	Reducerea istoricului la starea curentă.	Ce intrări acceptă și ce pierde?
Partiție	Istorii cu aceeași reprezentare.	Ce situații sunt confundate?
Predicat	Existența unei ordini complete.	Poate fi verificat înaintea execuției?
Graf	Dependențele dintre operații.	Ce trebuie calculat sau invalidat?
Ordine parțială	Compararea informației păstrate.	Care reprezentare o poate reproduce pe alta?

Vocabularul nu este ornament. El ne va permite să transformăm expresii precum „agentul vede altfel problema” în afirmații despre funcții, clase, programe și rezultate observabile.

CAPITOLUL 03

Programe, semantică, interpretare

Ce fixează o semantică

Sintaxa stabilește ce expresii sunt bine formate. Semantica stabilește cum sunt interpretate. Într-un limbaj de programare, ea poate descrie tranzițiile de stare, rezultatul unei execuții sau proprietățile pe care comenzile le conservă. Prin urmare, semantica nu trebuie redusă la un dicționar static de sensuri. Interpretarea abstractă oferă explicit multiple semantici și relații între ele [Cousot1977].

Să luăm o comandă care adaugă unu la un număr. Semantica trebuie să precizeze dacă numărul este un întreg matematic nelimitat sau un întreg reprezentat pe un număr fix de biți. La valoarea maximă reprezentabilă, răspunsurile pot diferi. A spune numai „înțelege adunarea” ascunde o alegere esențială de model.

O semantică operațională descrie pașii execuției. Una denotațională asociază programului un obiect matematic care îi descrie comportamentul. Una axiomatică exprimă relații între condiții de intrare și proprietăți de ieșire. Aceste perspective pot fi complementare. MRP nu le înlocuiește printr-un termen mai larg; le tratează ca instrumente disponibile agentului.

Ce adaugă pragmatica

O regulă semantică nu decide întotdeauna singură dacă merită aplicată, ce informație lipsește sau ce scop trebuie satisfăcut. Aici introducem nivelul pragmatic: alegerea regimului de interpretare, a contextului relevant și a criteriului de oprire. După alegere, regulile interpretării trebuie să rămână suficient de stabile pentru verificare.

Putem numi „semantică stabilizată” o interpretare al cărei domeniu și ale cărei obligații sunt fixate pentru o execuție. Este o convenție internă a cărții, nu o clasificare universal acceptată a filosofiei limbajului. Ea permite un principiu util: agentul poate schimba interpretarea între episoade de revizuire, dar nu poate modifica tacit sensul regulilor în mijlocul unei demonstrații.

De exemplu, un sistem poate trece de la întregi matematici la aritmetică pe 32 de biți. Aceasta nu reprezintă o simplă optimizare dacă schimbă rezultatele. Este schimbarea modelului; concluziile vechi trebuie reexaminate acolo unde diferența contează.

Interpretorul ca obiect

În modelul propus, un interpretor primește intrare, context, scop și stare internă. Produce un rezultat, o stare internă nouă și evidențe. Cuvântul „evidență” va desemna aici un suport verificabil: surse, teste, urme de calcul sau demonstrații, nu simpla convingere exprimată de agent.

Interpretează(intrare, context, scop, stare) = (rezultat, stareNouă, evidențe).

Această expresie este o interfață conceptuală, nu un algoritm complet. Câmpurile trebuie restrânse pentru fiecare interpretor concret. Un analizor de ordine poate cere un tabel

cu indici unici. Un verificator poate cere o formulă într-o logică precisă. Un generator poate accepta text, dar rezultatul său va avea inițial statut de propunere.

Contextul nu trebuie să devină un loc în care ascundem informație nelimitată. Dacă selectorul poate consulta textul integral, dar predictorul de referință primește numai un rezumat, comparația nu măsoară doar calitatea interpretoarelor. Orice experiment va declara exact ce poate citi fiecare componentă.

Contracte și compoziție

Un contract precizează precondiții, rezultate promise și efecte permise. Logica de tip Hoare exprimă relația dintre o precondiție, o comandă și o postcondiție. Corectitudinea parțială spune că, dacă execuția se termină și precondiția era adevărată, postcondiția este adevărată; terminarea necesită o obligație separată [SoftwareFoundations].

Considerăm o operație care selectează ultimul eveniment al unei resurse. Contractul poate cere ca istoricul să fie nevid, ordinea să fie totală și identificarea resursei neambiguă. Postcondiția spune că evenimentul returnat aparține resursei și nu este urmat de alt eveniment al aceleiași resurse. Această postcondiție poate fi verificată independent de modul în care operația a fost implementată.

Compoziția a două interpretoare cere mai mult decât potrivirea unor formate. Dacă primul întoarce „ultimul eveniment cunoscut”, al doilea nu poate presupune „ultimul eveniment care a existat în lume” fără ipoteza completitudinii istoricului. Acesta este un exemplu de incompatibilitate semantică într-o interfață aparent corectă tehnic.

Interpretoarele probabiliste cer contracte mai prudente. „A produs frecvent răspunsuri corecte” nu este o postcondiție logică universală. Putem descrie domeniul testat, distribuția de evaluare și incertitudinea performanței. Compoziția unor componente probabiliste nu transformă aceste rezultate în certitudine.

Învățarea ca interpretor

O procedură de învățare interpretează datele printr-o familie de modele și un criteriu de ajustare. Ea produce parametri, reguli sau programe. Astfel, poate fi inclusă în aceeași schemă de interfață. Totuși, acest lucru nu face toate operațiile identice: schimbarea unui rezultat temporar diferă de schimbarea bibliotecii pe care vor depinde sarcinile viitoare.

Vom separa starea de lucru de starea consolidată. O ipoteză nouă poate exista într-o ramură experimentală fără a deveni regulă acceptată. Promovarea ei presupune teste și o decizie explicită. Această separare permite agentului să exploreze fără a trata explorarea ca adevăr deja dobândit.

Ce rămâne stabil

O arhitectură capabilă să se modifice are nevoie de un nucleu suficient de stabil pentru a determina ce modificări au avut loc. Nucleul poate fi mic: administrarea versiunilor, regulile de execuție, tipurile de evidențe și contabilizarea costurilor. Nu trebuie să conțină dinainte toate conceptele domeniului.

MRP devine interesantă dacă nucleul permite dezvoltarea unor interpretoare utile fără rescrierea întregului sistem. Componentele furnizate manual sunt legitime, dar trebuie declarate separat de cele învățate.

CAPITOLUL 04

Probabilitate, decizie, adevăr

Incertitudinea nu este contradicție

Un agent poate să nu știe dacă există acces chiar dacă sistemul real are o stare precisă. Poate lipsi un eveniment, o sursă poate fi nesigură sau politica se poate fi schimbat. Necunoașterea stării și contradicția dintre două afirmații sunt situații diferite. Arhitectura trebuie să le poată reprezenta separat.

O probabilitate este o valoare între zero și unu atribuită unui rezultat într-un model. Pentru rezultate care se exclud reciproc și acoperă toate posibilitățile considerate, probabilitățile însumează unu. O distribuție este colecția acestor valori. Ea descrie incertitudinea agentului sau regularitatea unui mecanism, în funcție de modelul folosit.

Media unei variabile aleatoare nu este rezultatul care se va produce neapărat. Este suma rezultatelor posibile înmulțite cu probabilitățile lor, pentru un domeniu finit. Un rezultat cu probabilitate mică poate totuși apărea; un agent bine calibrat nu este unul ale cărui predicții individuale se adeveresc întotdeauna.

Actualizarea unei credințe

Să presupunem un exemplu fictiv: înaintea unui test, agentul atribuie probabilitate 0,2 existenței unei defecțiuni. Un semnal apare în 90% dintre cazurile cu defecțiune și în 10% dintre cele fără defecțiune. După apariția semnalului, probabilitatea actualizată este 0,18 împărțit la 0,26, adică aproximativ 0,692.

Calculul compară două moduri în care semnalul ar fi putut apărea: 0,2 × 0,9 din cazuri provin din defecțiuni, iar 0,8 × 0,1 din cazuri nu provin din defecțiuni. Noul rezultat este o probabilitate condiționată, nu o demonstrație. El depinde de corectitudinea ratelor inițiale și de aplicabilitatea lor situației curente.

Un interpretor MRP poate efectua această actualizare. Altul poate contesta modelul care furnizează ratele. Schimbarea numerelor într-un model și schimbarea modelului sunt două operații diferite; a doua poate necesita experiențe suplimentare, nu numai recalcularea primei.

Pierdere și alegere

O funcție de pierdere atribuie un cost unei decizii într-o situație. Dacă acordarea greșită a accesului produce o pierdere de 100, iar refuzul greșit produce o pierdere de 1, pragul rațional nu este probabilitatea 0,5. Presupunând cost zero pentru deciziile corecte și probabilitate p că accesul este legitim, acordarea are pierdere așteptată 100 × (1 − p), iar refuzul are pierdere p.

Acordarea este preferabilă numai când p depășește 100/101, aproximativ 0,9901. Acesta este un calcul despre o matrice de costuri fictivă. Nu este o recomandare de politică de acces. El arată de ce aceeași incertitudine poate conduce la decizii diferite sub obiective diferite.

Preferințele nu modifică însă distribuția adevărului. Pragul poate fi schimbat legitim; probabilitatea nu trebuie falsificată pentru a justifica decizia dorită. Un sistem transparent poate declara: „consider evenimentul probabil, dar costul unei erori cere verificare suplimentară”.

Evaluarea probabilităților

Regulile de scor proprii sunt construite pentru ca predicția probabilistică sinceră să optimizeze scorul așteptat. Gneiting și Raftery analizează această clasă de metode și diferența dintre calibrare și concentrarea informativă a predicțiilor [Gneiting2007]. Într-un experiment MRP, ele pot împiedica reducerea evaluării la simpla alegere a etichetei majoritare.

Pentru un rezultat binar y, egal cu zero sau unu, o variantă uzuală a pierderii Brier este $(p - y)^2$. Dacă probabilitatea reală este q, pierderea așteptată se poate rescrie drept $(p - q)^2 + q \times (1 - q)$. Al doilea termen nu depinde de predicția p. Primul este minim când $p = q$. Această derivare explică de ce scorul favorizează probabilități corecte, nu doar etichete corecte.

Scorul nu poate face datele reprezentative prin el însuși. Un agent evaluat numai pe situații ușoare poate avea un scor excelent și performanțe slabe exact în cazurile importante. Distribuția testelor face parte din specificația experimentului.

Abținere și informație

Abținerea este o acțiune cu un cost propriu: întârziere, intervenție umană sau oportunitate pierdută. Nu trebuie tratată nici ca eșec absolut, nici ca soluție gratuită. O tehnologie utilă va raporta simultan rata răspunsurilor, eroarea printre răspunsurile oferite și costul cazurilor redirecționate.

Cunoașterea mai precisă are valoare când poate schimba decizia într-un mod suficient de important. O observație nouă care nu modifică niciun comportament relevant poate avea valoare științifică, dar valoare operațională mică în episodul curent. Într-un program de cercetare, valoarea pe termen lung a învățării trebuie contabilizată separat de succesul imediat.

Pentru MRP, probabilitatea descrie ceea ce agentul estimează, pierderea exprimă ce contează pentru sarcină, iar verificarea stabilește ce suport există. Acestea sunt trei obiecte distincte. Păstrarea separării lor face posibil un metaraționament care economisește calcul fără a transforma economia într-o permisiune de a inventa fapte.

PARTEA II / Fundamente pentru distincții

CAPITOLUL 05

Reprezentări și echivalențe

Ce are voie să uite agentul

Orice reprezentare finită selectează. Ea păstrează unele diferențe și ignoră altele. Problema nu este eliminarea oricărei pierderi, ci identificarea pierderilor incompatibile cu sarcina. Un rezumat care omite culoarea copertei poate fi perfect adecvat unei analize a argumentelor. Același rezumat este insuficient pentru identificarea ediției după aspect.

Fixăm o familie T de teste. Un test poate fi o întrebare cu răspuns verificabil sau o procedură de interacțiune cu mediul. Pentru început, considerăm teste deterministe. Două istorii sunt echivalente relativ la T dacă fiecare test din T produce același răspuns pentru ambele istorii.

h1 echivalent-T h2 exact atunci când Test(h1) = Test(h2) pentru fiecare Test din T.

Relația este reflexivă, simetrică și tranzitivă deoarece egalitatea răspunsurilor are aceste proprietăți. Ea produce o partiție a istoriilor. Această partiție spune ce informație poate fi ignorată fără a modifica răspunsurile la testele declarate. Nu spune ce poate fi ignorat pentru orice întrebare viitoare imaginabilă.

Reprezentarea suficientă

O reprezentare R este suficientă pentru T dacă fiecare test poate fi rezolvat folosind numai R(h). Într-un domeniu finit, fără restricții de calcul, aceasta este echivalent cu cerința ca R să nu unească istorii care au răspunsuri diferite la un test relevant.

Demonstrația este directă. Dacă două istorii primesc aceeași reprezentare, orice funcție care vede numai reprezentarea va produce același rezultat pentru ele. Prin urmare, nu poate reproduce două răspunsuri diferite. Reciproc, dacă în fiecare clasă R toate răspunsurile sunt identice, putem asocia clasei acel răspuns. Astfel construim o funcție asupra reprezentărilor care reproduce testul.

Rezultatul este informațional. Funcția construită poate necesita un tabel foarte mare sau poate fi greu de calculat. Suficiența informației nu este același lucru cu eficiența accesului la ea. O arhivă brută poate conține tot ceea ce trebuie, dar o procedură specializată poate face informația utilizabilă într-un timp acceptabil.

Partiția indusă de T este cea mai grosieră partiție suficientă pentru aceste teste. Orice altă reprezentare suficientă trebuie să păstreze cel puțin distincțiile ei. Formularea nu implică unicitatea sintaxei: aceleași clase pot fi codificate prin numere, etichete, programe sau alte obiecte.

O limită demonstrabilă

Revenim la acordare și revocare. Presupunem că fiecare istoric este disponibil integral înainte de comprimare, că ultima operație determină starea și că reprezentarea păstrează numai mulțimea operațiilor. Cele două ordine primesc aceeași reprezentare. Dacă apar cu probabilități egale, niciun predictor care vede numai acea reprezentare nu poate depăși o acuratețe așteptată de 50% pe această pereche.

Un predictor determinist trebuie să aleagă un răspuns și greșește pe unul dintre cazuri. Un predictor aleator care spune „da” cu probabilitate p are acuratețe 0,5 × p + 0,5 × (1 − p), tot 0,5. Randomizarea nu reconstruiește ordinea pierdută.

Afirmația încetează să se aplice dacă predictorul vede o altă sursă de informație: memoria episodului, identificatorul cazului corelat cu răspunsul sau istoricul brut. De aceea, un experiment despre pierderea informației trebuie să controleze toate canalele de intrare, nu numai vectorul pe care autorul îl numește „reprezentare”.

Pentru un set finit etichetat, cea mai bună clasificare bazată exclusiv pe R alege eticheta majoritară în fiecare clasă R. Numărul minim de greșeli este numărul total de exemple minus suma frecvențelor majoritare din clase. Această limită poate fi calculată exact și oferă un instrument de diagnostic înaintea antrenării unui predictor mai complex.

Rafinarea ca învățare

Dacă o funcție nouă D separă cazurile confundate, putem defini Rnou(h) = (R(h), D(h)). Noua reprezentare păstrează informația veche și adaugă distincția. D poate fi o întrebare calculabilă: ultima operație, numărul anulărilor, existența unei excepții sau autorul unei afirmații.

Acesta este sensul expresiei „o dimensiune este un program”. Dimensiunea nu trebuie să fie un număr real fără interpretare. Poate fi un rezultat discret ori o structură relațională. Agentul învață o operație care schimbă ceea ce poate observa în propriile date.

Rafinarea poate să nu ceară observații externe noi, dar nu este gratuită computațional. Pentru a calcula D, poate fi necesară recitirea unei arhive lungi. Dacă arhiva a fost eliminată, D poate deveni imposibil de aplicat retrospectiv. Proiectarea memoriei determină ce fel de învățare ulterioară rămâne posibilă.

Ce nu garantează compresia

O reprezentare suficientă pentru întrebările de astăzi poate fi insuficientă pentru cele de mâine. Putem gestiona această limită prin păstrarea surselor, prin reprezentări multiple și prin posibilitatea de a cere observații noi. Nu o putem elimina printr-o afirmație universală că „agentul a înțeles esența”. Esența operațională trebuie raportată la o familie de utilizări.

Teoria informației distinge măsurarea și codarea informației de interpretarea semantică a mesajelor; această delimitare este explicită în lucrarea lui Shannon [Shannon1948]. MRP adaugă o întrebare de proiectare: ce informație trebuie conservată pentru testele și deciziile relevante? Un fișier foarte bine comprimat poate rămâne greu de folosit, iar o reprezentare redundantă poate permite răspunsuri rapide.

Prin urmare, criteriul final nu este „cât de mică este memoria?”, ci „ce competență se păstrează, la ce cost de construcție și utilizare, pentru ce distribuție de sarcini?”. Această întrebare va guverna atât învățarea interpretoarelor, cât și evaluarea lor.

CAPITOLUL 06

Învățarea stărilor

Un protocol mic

Considerăm un protocol fictiv cu trei stări: inactiv, în așteptare și activ. O solicitare mută sistemul din inactiv în așteptare. O confirmare activează numai o solicitare deja existentă. O anulare revine la inactiv. Pentru a evita ambiguitatea, o solicitare repetată nu dezactivează un acces deja activ, iar confirmarea unui acces activ îl lasă activ.

Agentul nu vede numele stării. Vede operațiile și poate întreba dacă accesul este activ. Două istorii, istoricul gol și istoricul care conține o solicitare, produc același răspuns negativ la întrebarea curentă. Totuși, după confirmare, răspunsurile devin diferite. Starea nu poate fi identificată numai prin rezultatul prezent; contează reacția la continuări.

Istoric reprezentativ	**Acces acum**	**Acces după confirmare**	**Stare necesară modelului**
Niciun eveniment	Nu	Nu	Inactiv
Solicitare	Nu	Da	În așteptare
Solicitare, confirmare	Da	Da	Activ

Acesta este un exemplu în care un nou test produce o nouă distincție. Agentul nu inventează arbitrar „încă o stare”; descoperă că două situații tratate identic au comportamente diferite sub o continuare executabilă.

Automat și memorie minimă

Un automat finit are un număr finit de stări și o regulă de tranziție pentru fiecare intrare admisă. Starea rezumă ceea ce trebuie reținut despre trecut pentru comportamentul viitor al modelului. Un automat nu este o explicație obligatorie a creierului, ci un obiect matematic potrivit pentru protocoale și procese discrete.

Principiul Myhill–Nerode identifică, pentru limbaje regulate, istorii care nu pot fi deosebite prin nicio continuare în raport cu acceptarea. În învățarea activă a automatelor, această perspectivă conduce la construirea stărilor din răspunsuri la interogări. Algoritmul L* al Danei Angluin este o referință fundamentală, iar generalizările către sisteme reactive și procese decizionale arată relevanța sa dincolo de simpla recunoaștere a șirurilor [Angluin1987; Tappler2019].

Pentru MRP, lecția nu este că toate conceptele trebuie reduse la automate. Lecția este că unele entități interne ale agentului pot fi construite din incapacitatea unor teste de a distinge istorii. O stare devine o ipoteză despre comportamentul viitor, nu o etichetă semantică atribuită manual.

De ce contează închiderea

Dacă două istorii sunt considerate echivalente, aplicarea aceleiași operații trebuie să le conducă în stări încă echivalente. Altfel, tranziția din clasa comună nu este bine definită. Această proprietate se numește congruență la dreapta în cazul continuărilor de șiruri.

Echivalența pentru toate continuările relevante are această proprietate: o continuare care ar distinge succesorii ar putea fi precedată de operația comună și ar distinge istoriile inițiale. Dar echivalența pentru o listă finită, aleasă arbitrar,

de teste nu o are obligatoriu. În protocolul nostru, testul „acces acum” unește inactivul cu așteptarea, deși confirmarea le separă.

Prin urmare, o bază de teste trebuie evaluată și pentru actualizare, nu numai pentru răspunsul imediat. O reprezentare care răspunde astăzi corect, dar nu poate fi actualizată corect după evenimentul următor, nu este o stare suficientă a sistemului dinamic.

O buclă de învățare

Agentul poate menține un tabel cu istorii reprezentative pe linii și continuări de test pe coloane. Linii identice sugerează aceeași stare. Când o nouă continuare produce răspunsuri diferite, liniile sunt separate. Când o operație duce la un tip de linie încă nereprezentat, trebuie introdus un reprezentant nou.

Acesta este un mecanism de construcție, nu doar o clasificare post-hoc. După fiecare extindere, agentul construiește un model candidat și îl confruntă cu sistemul. Un contraexemplu este o secvență pe care comportamentul candidatului diferă de comportamentul țintei. Secvența poate fi redusă pentru a localiza testul care lipsea.

Într-un cadru ideal, un oracol de echivalență spune dacă modelul candidat este corect și oferă un contraexemplu când nu este. Într-un sistem real, un asemenea oracol este de obicei înlocuit de testare. Lucrarea lui Tappler și colaboratorii separă explicit un cadru cu informație perfectă de învățarea prin eșantionarea traseelor [Tappler2019]. Absența unui contraexemplu găsit nu trebuie reinterpretată drept demonstrarea echivalenței.

Resetare și acces experimental

Pentru a compara două continuări pornind din aceeași situație, poate fi necesară resetarea sistemului sau reproducerea unui istoric. Într-un simulator aceasta poate fi simplă. Într-un flux organizațional, poate fi scumpă sau imposibilă. Capacitatea de experimentare nu trebuie presupusă gratuit.

Un mediu de învățare trebuie să declare ce acțiuni sunt disponibile, ce observații sunt obținute, dacă se poate reseta și dacă execuția modifică ireversibil lumea. Aceste condiții determină ce algoritmi pot fi aplicați și ce afirmații pot fi validate.

Ce câștigă MRP

Un automat învățat poate deveni un interpretor reutilizabil al istoricului. El poate procesa evenimente noi incremental, fără să recitească permanent întregul trecut. Dar această eficiență presupune că starea învățată păstrează diferențele necesare în domeniul evaluat.

Experimentul trebuie să distingă inducția unui model nou de selecția unuia furnizat. Numai prima testează construirea distincțiilor; ambele pot avea utilitate inginerească.

CAPITOLUL 07

Predicție și intervenție

Dincolo de stări vizibile

Într-un sistem probabilist, două istorii pot produce rezultate diferite la repetarea aceluiași test fără să reprezinte neapărat stări diferite. Diferența poate proveni din aleatoriu. Criteriul de echivalență trebuie să compare distribuții, nu o singură observație.

Vom nota Distribuție(test, h) distribuția rezultatelor testului după istoricul h. Două istorii sunt echivalente predictiv relativ la T dacă aceste distribuții sunt egale pentru fiecare test din T. Egalitatea exactă este un obiect teoretic; în experiment estimăm distribuțiile din eșantioane și raportăm incertitudinea.

Un test interactiv include modul de alegere a acțiunilor. Dacă prima observație determină acțiunea următoare, testul este o politică de experimentare, nu doar o listă fixă. Două sisteme pot fi comparate numai dacă folosim aceeași specificație a testului și aceleași condiții relevante de pornire.

Reprezentări predictive

Reprezentările predictive de stare, cunoscute ca PSR, descriu starea prin predicții despre rezultatele unor teste executabile asupra sistemului. Singh, James și Rudary dezvoltă această idee folosind o matrice a dinamicii sistemului, ale cărei linii corespund istoriilor, iar coloane testelor [Singh2004].

Intuiția se vede într-un exemplu. În loc să stocăm o etichetă ascunsă precum „modul latent 7”, putem stoca probabilitatea ca o verificare simplă să reușească, probabilitatea ca două operații succesive să reușească și probabilitatea ca un reset să restabilească funcționarea. Aceste coordonate au sens deoarece se referă la observații care pot fi colectate.

Nu orice listă de predicții este o PSR suficientă. Trebuie să existe o regulă pentru a calcula din ea predicțiile relevante și pentru a o actualiza după acțiuni și observații. Alegerea unui număr mic de întrebări intuitive nu asigură această proprietate.

Rangul fără mister

O matrice este un tabel numeric. Rangul său măsoară câte coloane sau linii independente sunt necesare pentru a descrie informația liniară din tabel. Dacă unele predicții pot fi obținute ca combinații liniare ale altora, nu trebuie stocate toate ca dimensiuni independente.

Aceasta oferă o posibilă compresie pentru sisteme cu structură adecvată. Totuși, estimarea unui rang mic din date finite nu demonstrează că sistemul real are exact acel rang. Zgomotul poate face coloane aparent independente, iar testele insuficiente pot ascunde diferențe. O reprezentare predictivă utilă cere atât o ipoteză structurală, cât și o procedură de validare.

Coeficienții unei combinații liniare nu trebuie interpretați automat ca probabilități. Ei pot fi negativi, deși predicția finală trebuie să rămână o probabilitate validă. În plus, actualizarea după observarea unui eveniment implică de regulă normalizare. Compresia liniară a predicțiilor nu înseamnă că întregul agent devine o simplă medie de modele.

A observa nu este a interveni

Considerăm două modele cu variabile binare. În primul, o variabilă ascunsă Z determină atât X, cât și Y. În al doilea, Z determină X, iar X determină Y. Presupunem că Z este zero sau unu cu probabilități egale și că toate legăturile copiază valoarea. În datele observaționale ale ambelor modele vedem mereu X = Y.

Dacă intervenim și fixăm X la zero, diferența apare. În primul model, Y continuă să copieze Z și este unu în jumătate dintre cazuri. În al doilea, Y copiază X și devine zero. Aceeași distribuție observațională poate fi compatibilă cu efecte diferite ale intervenției.

Formalismul cauzal pune în centru această separare între condiționare, intervenție și întrebări contrafactuale [Pearl2018]. Pentru MRP, consecința practică este că un interpretor antrenat pe corelații nu primește automat autoritatea de a prezice efectele oricărei acțiuni. Are nevoie de ipoteze cauzale sau de date experimentale adecvate.

O PSR a unui sistem controlat poate prezice teste de acțiune în condițiile sale de identificare și acoperire. Aceasta nu echivalează cu rezolvarea tuturor întrebărilor cauzale despre mecanisme ascunse ori despre lumi contrafactuale neobservate. Trebuie precizat exact ce intervenții au fost disponibile și ce clasă de predicții este susținută.

Identificabilitate și lipsuri

Un model este identificabil pentru o întrebare dacă datele și ipotezele disponibile determină răspunsul relevant. Două modele care explică toate observațiile pot rămâne diferite în privința unei acțiuni netestate. În această situație, rafinarea reprezentării interne nu produce singură informație nouă despre lume.

Agentul trebuie să aleagă între experimentare, presupunerea explicită a unui model sau abținere. Generarea unei explicații suplimentare nu este a patra cale de obținere a informației. Această limită împiedică transformarea MRP într-o retorică a omniscienței prin reorganizare internă.

Rolul în arhitectură

Interpretoarele predictive pot furniza coordonate verificabile, pot detecta coliziuni și pot sugera teste care ar separa modele candidate. Un meta-controler poate decide dacă beneficiul informației justifică experimentul. Un verificator poate controla dacă afirmația finală depășește domeniul susținut de date.

Astfel, MRP poate lega reprezentarea de acțiune fără a reduce toate formele de înțelegere la predicția următorului simbol. Predicția relevantă poate privi un protocol, o intervenție, un răspuns documentar sau consecința unei reguli. Specificarea acestei ținte este parte din problemă, nu un detaliu care poate fi lăsat implicit.

CAPITOLUL 08

Interpretări abstracte

Informație suficientă pentru o proprietate

Interpretarea abstractă construiește aproximări corecte ale comportamentului programelor. În loc să execute fiecare caz concret, operează într-un domeniu care păstrează anumite proprietăți și ignoră altele [Cousot1977]. Pentru MRP, aceasta oferă un exemplu matur de pluralitate a interpretoarelor cu obligații matematice clare.

Să presupunem că două variabile x și y sunt ambele egale cu același număr a, iar a este între zero și zece. O reprezentare prin intervale păstrează x între zero și zece și y între zero și zece. Aplicată independent celor două intervale, scăderea produce intervalul dintre minus zece și zece. Dar rezultatul concret x − y este întotdeauna zero.

Problema nu este că intervalele au fost calculate greșit. Ele au pierdut relația dintre variabile. O interpretare relațională care păstrează egalitatea x = y poate demonstra imediat rezultatul. Alegerea domeniului abstract determină ce proprietăți devin accesibile analizei.

Aproximare corectă

Într-o analiză prin supraaproximare, rezultatul abstract trebuie să includă toate rezultatele concrete posibile. Poate include și posibilități care nu apar în realitate. Acestea produc alarme false, nu neapărat erori ale analizei. O aproximare care exclude un comportament concret posibil poate deveni nesigură pentru proprietatea urmărită.

Putem vedea valoarea abstractă drept o descriere a unui set de stări concrete. „x este pozitiv” descrie multe valori. O transformare abstractă este corectă dacă, pornind de la toate stările descrise de intrare, nicio stare concretă posibilă după operație nu este pierdută din descrierea ieșirii.

Aceasta este o obligație mult mai puternică decât „operația a fost corectă pe exemplele testate”. Un interpretor învățat empiric nu devine un analizor abstract corect doar pentru că produce intervale sau formule. Trebuie verificată relația dintre operația concretă și transformarea abstractă.

Precizie și cost

Domeniile mai expresive pot demonstra proprietăți pe care domeniile simple le pierd, dar pot necesita mai mult calcul. Unele relații sunt importante într-o regiune a programului și inutile în alta. O strategie rezonabilă este să înceapă simplu și să mărească precizia acolo unde proprietatea cerută o impune.

Introducerea lui Cousot asupra interpretării abstracte explică această relație între proprietățile urmărite, aproximare și analiza calculabilă [CousotIntro]. Extensia MRP ar fi un controler care învață ce schimbare de reprezentare este probabil să rezolve o anumită alarmă, fără să presupună că „mai multă precizie” este întotdeauna gratuită sau necesară.

De exemplu, pentru verificarea că x nu depășește zece, intervalul poate fi suficient. Pentru verificarea egalității $x - y = 0$, relația este decisivă. Alegerea pragmatică depinde de întrebare; corectitudinea fiecărei interpretări depinde de regulile sale.

Puncte fixe și oprire

Programele cu bucle pot necesita propagarea repetată a informației. Un punct fix este o stare a analizei care nu se mai schimbă după încă o aplicare a regulii de propagare. Nu orice proces ajunge rapid la un punct fix, iar unele domenii permit lanțuri infinite de aproximări tot mai precise sau tot mai largi.

Interpretarea abstractă folosește mecanisme de accelerare, inclusiv lărgirea, pentru a forța stabilizarea unor analize [CousotIntro]. Prețul poate fi pierderea preciziei. Pentru MRP, ideea generală este că o politică de oprire trebuie să fie parte din algoritm. „Continuă să rafinezi până înțelegi” nu definește o procedură cu resurse finite.

Un prototip poate evita inițial domeniile complicate și poate folosi tabele finite ori limite explicite de pași. Restricția este legitimă dacă domeniul experimentului o declară. Ea devine problematică numai când concluziile sunt extinse tacit la sisteme fără aceleași limite.

Două forme de evidență

Un rezultat formal poate spune: sub aceste ipoteze despre program și aritmetică, proprietatea este satisfăcută pentru toate execuțiile modelate. Un rezultat empiric poate spune: în această distribuție de teste, interpretorul a avut o anumită rată de eroare. Ambele sunt utile, dar nu sunt interschimbabile.

O arhitectură hibridă poate permite unui generator să propună o abstracție și unui verificator să îi controleze corectitudinea locală. Dacă verificarea eșuează, ipoteza nu trebuie promovată ca regulă sigură. Dacă verificarea este incompletă, statutul rezultatului trebuie să păstreze această limită.

Transferul către MRP

Interpretarea abstractă arată că „mai multe moduri de interpretare” nu este o idee nouă în sine. Arată și cum pluralitatea poate fi disciplinată prin relații formale. MRP merită investigată acolo unde adaugă un mecanism măsurabil de alegere, construcție și reutilizare a acestor moduri.

Prima aplicație plauzibilă nu este un agent care își inventează propria matematică fără control. Este un agent care identifică ce relație lipsește unei analize, propune un interpretor adecvat și demonstrează sau testează exact ce îmbunătățire a obținut. Acest obiect este mai modest, dar suficient de dificil și de util pentru un program de cercetare real.

PARTEA III / Un mecanism de învățare

CAPITOLUL 09

Diagnosticul erorilor

De ce nu orice eșec cere o nouă reprezentare

Un agent răspunde greșit. Ce trebuie schimbat? Dacă fiecare eroare duce la construirea unui interpretor, biblioteca devine rapid o colecție de excepții. Dacă nicio eroare nu poate modifica reprezentările, agentul rămâne prizonierul distincțiilor proiectate inițial. Diagnosticul este veriga dintre aceste două excese.

În modelul propus, un incident este păstrat împreună cu intrarea, reprezentarea efectiv utilizată, versiunea interpretorului, predicția, rezultatul observat și condițiile observației. Diferența dintre predicție și rezultat nu stabilește singură cauza. Ea deschide o investigație delimitată.

Cauză posibilă	**Exemplu**	**Răspuns adecvat**
Eroare de calcul	Ordinea există, dar selecția ultimului eveniment este defectă	Corectarea implementării
Pierdere de reprezentare	Istorii diferite devin aceeași mulțime de evenimente	Rafinarea reprezentării
Informație neobservată	Revocarea nu a fost transmisă agentului	O nouă observație sau abținere
Variație aleatoare	O operație reușește numai probabilistic	Estimarea distribuției
Schimbarea regulilor	O nouă versiune modifică prioritatea evenimentelor	Detectarea schimbării și versionare
Specificație ambiguă	„Acces” poate însemna drept formal sau conexiune activă	Clarificarea țintei

Aceste cauze pot coexista. Un interpretor defect poate ascunde și o lipsă de informație. Diagnosticul trebuie deci să rămână revizuibil, nu să fie o etichetă definitivă atribuită de un clasificator.

Coliziunea ca martor al insuficienței

O coliziune relevantă apare când aceeași reprezentare corespunde unor răspunsuri incompatibile pentru același test și aceleași condiții. Pentru a o identifica trebuie verificate tocmai aceste condiții. Nu este o coliziune dacă unul dintre exemple folosește o altă regulă sau dacă întrebările sunt diferite.

Într-o microlume deterministă, două istorii pot furniza un martor exact. Într-un mediu stocastic, două rezultate diferite nu sunt suficiente: aceeași distribuție poate produce ambele rezultate. Acolo trebuie demonstrată sau estimată o diferență de distribuții, cu suficiente repetiții și cu controlul condițiilor relevante.

O procedură practică ar căuta mai întâi coliziuni în jurnalul de execuții. Apoi ar încerca să reproducă perechea într-un mediu controlat. Dacă reproducerea confirmă diferența, agentul ar genera ipoteze despre informația pierdută. Dacă nu o confirmă, incidentul rămâne deschis sau este reclasificat.

Rafinarea ghidată de contraexemple

CEGAR, rafinarea abstracțiilor ghidată de contraexemple, furnizează o disciplină importantă: o abstracție este verificată, iar un contraexemplu abstract este confruntat cu comportamentul concret. Dacă acel contraexemplu este artificial, abstracția poate fi rafinată pentru a-l elimina [Clarke2000].

MRP ar prelua ideea de buclă, nu ar pretinde că orice eroare predictivă este același obiect matematic. În verificare, o alarmă falsă poate cere mai multă precizie. În învățare, o

predicție falsă poate cere date noi, o distribuție diferită sau corectarea unui program. Identificarea tipului de eșec precedă alegerea rafinării.

Un candidat la rafinare trebuie să explice de ce reprezentarea actuală confundă cazurile. „Adaugă încă o componentă” nu este o explicație suficientă. „Păstrează ultima operație pentru fiecare pereche persoană–resursă, deoarece ordinea operațiilor schimbă răspunsul” este o ipoteză operațională.

Distincții minimale și distincții generale

Programul „recunoaște exact istoricul care a eșuat” separă orice caz memorat. Nu este însă, de obicei, distincția căutată. Un program care urmărește ultima operație relevantă poate separa o familie întreagă de cazuri. Diferența dintre cele două nu se vede pe incidentul inițial, ci pe exemple noi.

De aceea, propunerea unei distincții trebuie urmată de generarea unor cazuri contrastive. Se schimbă identitatea persoanei, se adaugă evenimente irelevante, se repetă o operație, se intercalează alte resurse. Programul ar trebui să păstreze ceea ce contează și să ignore variațiile nerelevante pentru regula studiată.

Aceste transformări nu sunt arbitrare. Dacă repetarea unei operații schimbă sensul protocolului, nu poate fi tratată ca invariabilitate. Familia transformărilor acceptate face parte din specificația testului. Chiar și construirea testelor poate introduce presupuneri greșite.

Informația imposibil de recuperat

Să presupunem că istoricul a fost eliminat și că a rămas doar un set fără ordine. Dacă două istorii valide produc același set, niciun program care citește numai acel set nu poate

reconstrui sigur care istorie s-a petrecut. Putem estima o probabilitate din frecvențe, dar aceasta nu este recuperarea evenimentului real.

MRP trebuie să poată răspunde: „am identificat distincția necesară, dar nu mai am informația care ar permite calcularea ei". Această stare este un rezultat util. Ea poate determina schimbarea politicii de memorare sau solicitarea unui nou eveniment, fără a fabrica o soluție retrospectivă.

Într-un sistem implementabil, diagnosticul ar produce una dintre trei ieșiri majore: o reparație calculabilă din informația existentă, un experiment care poate furniza informația lipsă sau o limită explicită. Promovarea tuturor incidentelor în prima categorie ar transforma arhitectura într-un generator de explicații neverificabile.

Ce se poate măsura

Calitatea diagnosticului are propriul set de teste. În microlumi putem injecta separat erori de implementare, coliziuni, observații lipsă și schimbări de reguli. Măsurăm cât de des agentul identifică mecanismul potrivit și cât costă investigația.

Acest test este diferit de acuratețea finală. Un sistem poate ajunge uneori la răspunsul corect printr-o reparație greșită, care va eșua ulterior. Pentru MRP contează dacă îmbunătățirea este legată de cauza reală a dificultății, nu numai dacă un scor punctual crește.

CAPITOLUL 10

Inducția interpretoarelor

Ce se învață și ce este furnizat

Un prim sistem MRP nu trebuie să inventeze limbajul mașinii, logica sau toate operațiile de bază. El poate primi un nucleu de funcții și poate învăța compoziții noi. Contribuția învățării trebuie descrisă exact: ce primitive existau, ce structuri puteau fi exprimate și ce programe au fost descoperite.

Pentru istorii de evenimente, primitivele pot fi filtrarea după un câmp, compararea a două valori, alegerea unui extrem, numărarea, asocierea după o cheie și aplicarea unei reguli de tranziție. Un tip limitează intrările admisibile: selectarea ultimei înregistrări primește o secvență ordonabilă, nu o probabilitate sau o propoziție izolată.

Tipurile reduc căutarea și previn unele erori, dar nu demonstrează intenția. Un program poate fi bine tipizat și să aleagă prima înregistrare în locul ultimei. Corectitudinea structurală este o condiție necesară pentru anumite operații, nu certificatul întregii soluții.

Căutarea într-un limbaj finit descris

Inducția poate începe prin enumerarea compozițiilor scurte. Dacă avem b opțiuni aproximativ la fiecare poziție și permitem lungimea d, numărul de secvențe candidate poate crește de ordinul b la puterea d. Tipurile, simetriile și exemplele elimină o parte din combinații, dar nu anulează dificultatea combinatorică.

Un generator ghidat poate prioritiza programele care par potrivite incidentului. De exemplu, diferența dintre două istorii cu aceleași elemente, dar în altă ordine, sugerează operații sensibile la ordine. Această euristică nu garantează succesul; oferă un mod verificabil de a reduce calculul.

Un LLM poate participa ca generator de asemenea ipoteze. Avantajul potențial este propunerea rapidă a unor compoziții care ar fi costisitor de enumerat. Dezavantajul este că el poate furniza informație suplimentară din antrenare, poate produce programe fragile și poate domina contribuția întregului sistem. Evaluarea trebuie să includă o variantă cu același LLM, aceleași instrumente și același buget, dar fără mecanismul MRP de consolidare.

Un criteriu de selecție, nu doar potrivirea pe exemple

O funcție de selecție poate combina eroarea, complexitatea descrierii și costul de execuție. Nu există ponderi universal corecte. Ele exprimă compromisul ales pentru aplicație și trebuie publicate împreună cu rezultatele.

Scor(candidat) = eroare estimată + penalizare a descrierii + cost estimat.

Principiul lungimii minime a descrierii, MDL, oferă un fundament pentru preferarea unei explicații care comprimă atât modelul, cât și datele rămase neexplicate [Grunwald2004]. În MRP, această preferință ar putea favoriza o regulă reutilizabilă în locul unei liste de excepții. Totuși, rezultatul depinde de limbajul în care măsurăm descrierea și de calitatea datelor. Un program scurt nu este automat adevărat sau etic preferabil.

Separarea dintre costul de descriere și cel de execuție este importantă. O definiție foarte scurtă poate ascunde o căutare enormă. Invers, un program mai lung poate precompila operații și poate rula rapid. Biblioteca trebuie evaluată la nivelul costului total pe sarcinile relevante.

Din program local în concept reutilizabil

Să presupunem că, în mai multe protocoale, agentul descoperă aceeași schemă: grupează după identitate, selectează ultimul eveniment și interpretează tipul acestuia. Consolidarea extrage partea comună și o parametrizează prin câmpurile și regulile specifice protocolului.

Această schemă este un candidat pentru o noțiune operațională de concept: o compoziție care organizează multe cazuri și poate fi executată în contexte noi. Nu pretindem că toate conceptele umane sunt astfel de programe. Definim un tip de cunoaștere pe care îl putem construi și măsura.

DreamCoder arată că învățarea de biblioteci de abstracții programatice și ghidarea căutării pot fi combinate într-un sistem concret [Ellis2020]. Pentru MRP, întrebarea distinctă ar fi dacă diagnosticul unei coliziuni poate ghida această inducție și dacă noua abstracție îmbunătățește explicit familia de distincții disponibile agentului.

Echivalența programelor și rescrierea

Biblioteca poate conține programe diferite care calculează același rezultat. Eliminarea redundanței ar economisi memorie și ar simplifica selecția. Dar stabilirea echivalenței pentru programe generale nu este o operație gratuită și universal decidabilă [Barak].

Putem lucra cu identități dovedite într-un limbaj restrâns. Saturarea prin egalități, inclusiv structuri precum cele utilizate de egg, permite reprezentarea compactă a multor expresii echivalente și alegerea uneia după un cost [Willsey2021]. Aceasta nu autorizează declararea echivalenței unor programe arbitrare produse de un LLM.

Dacă operațiile au efecte, dacă aritmetica produce depășiri sau dacă ordinea contează, regulile de rescriere trebuie să includă aceste condiții. Identitatea algebrică a expresiilor ideale poate să nu fie validă pentru execuția concretă. Într-un prototip, un nucleu pur și explicit limitat ar simplifica enorm verificarea.

Validare independentă și contaminare

Cazul care a generat ipoteza nu poate fi principala dovadă că ipoteza generalizează. Nici un set de teste consultat repetat, până când candidatul îl trece, nu mai funcționează ca evaluare independentă. Cercetarea asupra analizei adaptive a datelor arată de ce reutilizarea rezultatelor de evaluare poate compromite generalizarea [Dwork2015].

Aș separa exemplele de inducție, testele de dezvoltare și un set final inaccesibil selecției. Un evaluator ar putea returna numai verdictul necesar pentru etapa curentă, păstrând o rezervă pentru evaluarea finală. În plus, ar trebui raportate toate încercările relevante, nu numai ultimul candidat.

Inducția reușită nu înseamnă „a fost generat un program”. Înseamnă că un program construit din primitive declarate a reparat o insuficiență identificată, a trecut evaluări adecvate și a justificat costul consolidării. Acesta este obiectul experimental complet.

CAPITOLUL 11

Raționalitatea calculului

A calcula este tot o acțiune

Un agent are de ales nu numai între acțiuni în lume, ci și între operații interne. Poate consulta un tabel, poate executa un interpretor mai precis, poate cere o nouă observație sau poate construi o ipoteză. Fiecare opțiune consumă timp și poate schimba decizia finală.

Valoarea calculului este câștigul așteptat obținut printr-o operație internă, după scăderea costului său. Pentru o singură operație urmată de decizie, putem scrie fără a presupune o soluție perfectă:

ValoareCalcul = câștig decizional așteptat după calcul − câștig decizional acum − cost calcul.

Această formulă nu este o nouă lege MRP. Metaraționamentul și învățarea politicilor de selecție a calculelor au deja formulări experimentale, inclusiv în lucrarea lui Callaway și colaboratorii [Callaway2018]. MRP ar extinde repertoriul operațiilor selectabile la construirea și consolidarea unor interpretoare.

Un calcul care merită

Presupunem o stare ascunsă cu două valori echiprobabile. Un răspuns corect valorează o unitate, unul greșit zero. Fără informație, cea mai bună probabilitate de succes este 0,5. Un test perfect costă 0,1 unități din aceeași scară de utilitate.

După test, succesul este sigur, iar valoarea netă este 0,9. Câștigul față de răspunsul imediat este 0,4. Testul merită în modelul dat. Dacă testul ar costa 0,6, executarea lui ar reduce utilitatea totală. Aceeași informație poate fi rațional de obținut într-o aplicație și nerațional într-alta.

În practică, testele nu sunt perfecte și valoarea lor nu este cunoscută. Agentul trebuie să estimeze probabilitatea rezultatelor și efectul lor asupra deciziei. Estimarea valorii este ea însăși un calcul, deci trebuie să aibă un buget.

De ce selecția miopă poate eșua

Considerăm doi biți ascunși, independenți și echiprobabili. Sarcina este să răspundem dacă sunt diferiți. Fără observații, acuratețea maximă este 0,5. Aflarea unui singur bit nu îmbunătățește această acuratețe: celălalt rămâne necunoscut. Aflarea ambilor o ridică la unu.

Dacă fiecare observație costă 0,1, o politică ce evaluează numai operații individuale le va refuza pe ambele: fiecare costă fără să îmbunătățească imediat decizia. O politică ce evaluează perechea vede un câștig net de 0,3. Exemplul este calculat, nu un rezultat de simulare.

Compoziția interpretoarelor poate produce exact această complementaritate. O operație identifică entități, alta relații; separat nu răspund întrebării. De aceea, un controler MRP ar trebui testat cu diferite orizonturi de planificare. Nu trebuie presupus că o regulă locală rezolvă toate dependențele.

Cum evităm regresia meta

Dacă fiecare alegere cere o altă alegere despre cum să alegem, arhitectura nu se mai oprește. Soluția inginerească nu este un interpretor infinit de înțelept, ci un controler limitat: un număr maxim de evaluări, o politică aproximativă și o regulă de oprire.

La început, selecția poate fi o combinație între verificarea precondițiilor și o estimare empirică a succesului pe familii de sarcini. Mai târziu, politica poate fi învățată. Chiar și atunci trebuie comparată cu o regulă simplă. Complexitatea controlerului se justifică numai dacă produce un avantaj măsurabil.

Cercetarea recentă aplică raționalitatea metacalculului și modelelor lingvistice [DeSabbata2025]. Prin urmare, simpla selecție între un răspuns rapid și unul costisitor nu constituie contribuția distinctă MRP. Candidatul la contribuție este legătura dintre selecție, diagnosticul reprezentării și acumularea unor proceduri noi.

Orizontul reutilizării

Construirea unui interpretor poate fi nerentabilă pentru sarcina curentă, dar rentabilă pentru multe sarcini viitoare. Aceasta introduce un orizont de amortizare. Valoarea estimată depinde de frecvența cu care va reapărea problema, de stabilitatea regulilor și de costul verificării.

Un exemplu ipotetic: inducția și verificarea costă împreună 1.000 de milisecunde. Soluția veche costă 40 de milisecunde pe sarcină. Cea nouă costă opt, plus două pentru selecție. Economisirea este de 30 de milisecunde pe utilizare. Investiția se amortizează după 34 de utilizări, dacă toate celelalte condiții rămân identice.

Nu este suficient să raportăm că noul interpretor este de cinci ori mai rapid la execuție. Trebuie incluse costul construirii, rata de reutilizare și cazurile în care schimbarea regulilor îl invalidează. Avantajul unei biblioteci este unul pe durata de viață, nu numai pe ultimul apel.

Limite normative ale optimizării

O politică de calcul poate maximiza utilitatea dată, dar nu decide singură ce obiective sunt legitime. Costul unei erori pentru o persoană nu poate fi introdus tacit ca un număr ales de dezvoltator. Unele aplicații cer constrângeri care nu pot fi schimbate doar pentru a crește un scor mediu.

MRP ar trebui să distingă obiectivul tehnic, limitele aprobate și preferințele încă neclarificate. Într-un context cu consecințe importante, abținerea și intervenția umană pot fi acțiuni valide. O arhitectură meta-rațională nu devine responsabilă doar fiindcă își optimizează riguros propriul criteriu.

CAPITOLUL 12

Memorie și transfer

Trei memorii cu roluri diferite

Arhitectura propusă ar separa memoria episoadelor, memoria reprezentărilor și biblioteca de interpretoare. Episoadele păstrează ce s-a observat. Reprezentările păstrează rezultatul unor transformări. Biblioteca păstrează procedurile care pot reconstrui, consulta sau modifica aceste rezultate.

Separarea nu impune trei baze de date fizice. Ea impune posibilitatea de a răspunde la trei întrebări: „ce informație a fost disponibilă?”, „cum a fost comprimată?” și „ce operație a produs această concluzie?”. Fără aceste răspunsuri, o eroare de reprezentare se confundă ușor cu una de observație.

Memoria brută integrală este adesea costisitoare sau nepotrivită. Dar comprimarea ireversibilă limitează reinterpretarea. MRP ar trebui să trateze politica de păstrare ca pe o alegere explicită, cu clase de informații recuperabile și irecuperabile, nu ca pe un detaliu al infrastructurii.

Proveniență fără promisiuni excesive

Un rezultat ar trebui legat de versiunea intrării, de interpretor, de parametri și de activitatea care l-a produs. Modelul PROV al W3C oferă un vocabular pentru entități, activități, agenți și derivări [W3C2013]. El poate inspira interoperabilitatea jurnalului fără a obliga toate componentele interne la aceeași reprezentare.

Proveniența nu garantează adevărul. O afirmație falsă poate avea o istorie perfect documentată. Ceea ce obținem este posibilitatea de a inspecta dependențele, de a atribui transformările și de a determina ce rezultate trebuie reconsiderate după o schimbare.

Două documente care repetă aceeași sursă nu sunt automat două dovezi independente. O bibliotecă de proveniență poate arăta dependența comună. Agregarea încrederii trebuie să țină seama de ea, nu să numere pur și simplu referințele.

Transferul ca test al abstracției

Există cel puțin trei niveluri de reutilizare. Reexecutarea aceluiași program pe alte nume este un nivel modest. Refolosirea unei compoziții pe o structură nouă, cu alte combinații de reguli, este mai puternică. Transferul la un domeniu cu alte observații și alte condiții de valabilitate este și mai dificil.

Un interpretor care selectează ultima operație poate funcționa atât pentru o permisiune, cât și pentru o preferință actualizabilă. Nu rezultă că funcționează pentru un sold bancar, unde toate operațiile relevante pot contribui cumulativ. Asemănarea superficială a jurnalelor nu justifică transferul regulii.

Un contract de aplicare ar trebui să precizeze această diferență: proprietatea este determinată de ultima operație sau de agregarea operațiilor? Acesta este chiar un posibil nivel meta de învățare. Agentul poate învăța ce familie de interpretoare este potrivită unei clase de procese.

Consolidare, uitare și migrare

Adăugarea de interpretoare crește capacitatea, dar și costul căutării. Consolidarea poate fuziona definiții redundante, poate crea parametri și poate retrage variante inferioare. Retragerea nu trebuie să șteargă istoria rezultatelor care depind de ele. O versiune veche poate rămâne necesară pentru reproducerea unei analize.

Un criteriu de păstrare poate combina utilizarea recentă, valoarea estimată, unicitatea proprietăților demonstrate și costul reconstrucției. Acest criteriu trebuie testat pentru uitare catastrofală: pierderea unei capacități rare, dar importante, nu este justificată automat de frecvența ei redusă.

Migrarea unei reprezentări cere o transformare explicită. Dacă noul interpretor are nevoie de ordinea evenimentelor, iar vechea memorie nu o păstrează, migrarea nu poate fi completă. Sistemul trebuie să marcheze segmentele reconstruite, estimate și necunoscute.

Învățarea de lungă durată

Performanța unei biblioteci adaptive trebuie urmărită în timp. O curbă de învățare poate măsura costul mediu pe sarcină pe măsură ce biblioteca crește. O curbă de retenție măsoară dacă sarcinile vechi rămân rezolvabile. O curbă de transfer măsoară avantajul pe familii noi.

Aceste curbe pot avea forme diferite. Agentul poate deveni mai rapid pe sarcini cunoscute, dar mai slab pe cele noi, deoarece selectorul favorizează excesiv rutina. Sau poate generaliza mai bine, dar să consume memorie nesustenabilă. MRP trebuie evaluată ca sistem dinamic, nu printr-un singur număr după antrenare.

Cunoașterea acumulată nu ar fi, în această viziune, numai un depozit de propoziții. Ar fi un ansamblu de proceduri, rezultate, condiții și evidențe care permit agentului să refacă o distincție atunci când devine relevantă. Utilitatea acestei organizări rămâne o ipoteză empirică, dar componentele ei sunt suficient de precise pentru implementare.

PARTEA IV / Formele cunoașterii

CAPITOLUL 13

Distanțe pragmatice

Apropiere în raport cu o întrebare

Două descrieri pot folosi cuvinte foarte asemănătoare și pot cere acțiuni opuse. „Permis până vineri” și „interzis până vineri” sunt apropiate lexical, dar diferite pentru decizia de acces. Invers, două formulări foarte diferite pot descrie aceeași regulă. O distanță între texte nu este automat o distanță între consecințele lor.

Pentru MRP, întrebarea de pornire este: cât de diferite sunt rezultatele testelor relevante pentru două istorii? Aceasta fixează atât scopul distanței, cât și limitele sale. Nu definim o asemănare universală a lucrurilor; definim o asemănare în raport cu o familie declarată de experimente.

Variația totală

Pentru două distribuții discrete P și Q asupra acelorași rezultate, distanța de variație totală este jumătate din suma diferențelor absolute dintre probabilitățile lor. Ea este și cea mai mare diferență dintre probabilitățile atribuite aceluiași eveniment.

TV(P, Q) = 0,5 × suma, peste rezultate, a valorilor |P(rezultat) − Q(rezultat)|.

Dacă P și Q sunt distribuții ale unui răspuns binar, distanța este diferența absolută dintre probabilitățile răspunsului „da”. Predicțiile 0,2 și 0,7 sunt la distanța 0,5. Dacă sunt identice pentru toate rezultatele, distanța este zero.

Fixăm acum o familie de teste T. Pentru fiecare test, cele două istorii induc două distribuții asupra rezultatului. Distanța pragmatică propusă este cea mai mare diferență dintre aceste distribuții, peste testele familiei.

dT(h1, h2) = supremul, pentru t din T, al TV(Pt(. | h1), Pt(. | h2)).

Supremul înseamnă cea mai mică limită superioară. Pentru o familie finită de teste, el este pur și simplu maximul. Dacă un test include acțiuni, distribuțiile sunt cele ale executării acelui test în modelul declarat, nu condiționări observaționale interpretate tacit ca intervenții.

De ce este o pseudometrică

Distanța este nenegativă și simetrică. Este zero între o istorie și ea însăși. Inegalitatea triunghiului rezultă din inegalitatea corespunzătoare pentru fiecare test, urmată de luarea supremului. Anexa matematică oferă demonstrația pe pași.

Totuși, istorii distincte pot avea distanța zero: familia T poate să nu le distingă. De aceea numim funcția pseudometrică. Dacă grupăm toate istoriile cu distanță zero, obținem o metrică pe clasele rezultate. Această construcție face legătura dintre distanță și echivalențele din capitolul 5.

Metricile comportamentale și distanțele bazate pe bisimulare au antecedente importante în studiul proceselor decizionale [Ferns2004]. Distanța formulată aici nu trebuie confundată cu toate aceste metrici: alegerea testelor, a recompenselor și a orizontului schimbă obiectul matematic.

Aproximarea nu produce automat clase

În practică, nu cerem adesea distanță exact zero, ci o diferență sub un prag. Apare o capcană: relația „distanța este cel mult pragul” nu este, în general, tranzitivă.

Pentru trei distribuții binare cu probabilitățile 0,00, 0,06 și 0,12, primele două sunt apropiate la pragul 0,08, la fel ultimele două. Prima și ultima nu sunt. Dacă un algoritm unește toate perechile apropiate prin lanțuri, poate produce o clasă cu membri mult mai diferiți decât pragul inițial.

Prin urmare, gruparea aproximativă cere un criteriu suplimentar: diametru maxim al clasei, un centru cu rază controlată sau o limită explicită a erorii introduse de agregare. Formularea „reprezentăm toate stările similare prin același concept” este incompletă fără acest criteriu.

Când distanța controlează decizia

Presupunem că rezultatul unui test determină o utilitate între zero și unu. Diferența dintre utilitățile medii sub P și Q este cel mult TV(P, Q). Astfel, o distanță predictivă mică poate limita eroarea unei evaluări decizionale, dacă utilitatea este într-adevăr calculată din rezultatele modelate.

Un al doilea rezultat simplu este util. Dacă estimarea valorii fiecărei acțiuni este la distanță de cel mult e față de valoarea reală și alegem acțiunea cu estimarea maximă, pierderea față de acțiunea realmente optimă este de cel mult 2e. Un termen e apare din subestimarea acțiunii optime, iar celălalt din supraestimarea celei alese.

Aceste limite nu se transferă automat de la un singur pas la orice plan lung. Pentru acțiuni secvențiale trebuie controlate distribuțiile relevante ale traiectoriilor sau propagarea erorilor. O predicție bună a următorului eveniment nu garantează singură o politică bună pe termen lung.

Estimarea distanței costă

Distanța exactă presupune cunoașterea distribuțiilor. Din date finite obținem estimări și incertitudine. Un agent care declară două istorii identice fiindcă nu a găsit încă diferențe poate confunda lipsa dovezii cu dovada echivalenței.

Pentru primul experiment aș folosi microlumi în care evaluatorul cunoaște distribuțiile reale, iar agentul primește numai eșantioane. Astfel putem măsura cât de bine estimează agentul apropierea și când declară prea devreme suficiența unei reprezentări.

Rolul distanței este să transforme o metaforă despre „vecinătatea sensurilor” într-o afirmație verificabilă: aceste situații au consecințe apropiate pentru aceste teste, cu această eroare estimată. Abia ulterior merită căutată o reprezentare geometrică eficientă a acestor relații.

CAPITOLUL 14

Conuri și coordonate

Ce numim geometrie

O reprezentare geometrică asociază obiectelor coordonate și folosește relațiile dintre coordonate pentru a exprima o structură relevantă. Alegerea distanței, a dimensiunilor și a transformărilor admise face parte din model. Nu există o singură geometrie implicită a unei colecții de texte, experiențe sau programe.

Teoria spațiilor conceptuale a lui Gärdenfors propune descrierea conceptelor prin dimensiuni de calitate și regiuni geometrice [Gardenfors2000]. Ea motivează întrebarea despre forma conceptelor, dar nu dovedește că orice cunoaștere trebuie să fie convexă într-un spațiu fix. Pentru MRP trebuie precizat ce structură vrem să păstrăm: asemănare, ordine, comportament sau preferință decizională.

Convexitate și conuri

O mulțime este convexă dacă segmentul dintre oricare două puncte ale sale rămâne în interior. Un disc și un cub sunt exemple familiare. Un inel nu este convex: segmentul dintre două puncte opuse poate traversa golul central.

Un con, în sensul folosit aici, este închis la înmulțirea coordonatelor cu un număr nenegativ. Dacă este și convex, combinațiile nenegative ale punctelor sale rămân în con. Un con nu trebuie imaginat numai ca obiectul circular din geometria școlară; poate fi definit prin inegalități liniare în multe dimensiuni [Boyd2004].

Dimensionalitatea mare nu implică neconvexitate. Există mulțimi convexe în orice dimensiune finită. De asemenea, un con poate fi convex. A opune „conuri” unor „forme convexe” ar amesteca două clasificări diferite.

O derivare decizională

Presupunem un număr finit de stări posibile ale lumii. Agentul are o distribuție p asupra lor. Fiecare opțiune i este o procedură complet specificată: executarea unui interpretor, urmată de o regulă fixată de alegere a acțiunii pe baza rezultatului. Pentru fiecare stare, procedura are o utilitate netă u_i, care include costurile modelate.

Cuvântul „fixată” este esențial. Dacă procedura internă se schimbă arbitrar în funcție de p, utilitatea ei nu mai este reprezentată, în general, printr-un singur vector constant. Putem include mai multe proceduri candidate, dar nu putem păstra derivarea liniară ignorând această schimbare.

Utilitatea medie a procedurii i este produsul scalar dintre p și u_i: suma probabilității fiecărei stări înmulțite cu utilitatea în acea stare. Procedura i este preferabilă procedurii j când acest produs este cel puțin la fel de mare.

Regiune(i) = {p: toate coordonatele sunt nenegative, suma lor este 1, iar p · (u_i − u_j) ≥ 0 pentru fiecare j}.

Fiecare comparație definește un semispațiu. Intersecția lor cu simplexul probabilităților este o regiune convexă poliedrală. Simplexul este mulțimea tuturor distribuțiilor asupra stărilor enumerate: un segment pentru două stări, un triunghi pentru trei și un analog de dimensiune mai mare în general.

Trecem acum la ponderi nenormalizate z, toate nenegative. Condițiile de preferință devin omogene: înmulțirea tuturor ponderilor cu același număr pozitiv nu schimbă procedura preferată.

Con(i) = {z: z ≥ 0 și z · (u_i − u_j) ≥ 0 pentru fiecare j}.

Această mulțime este un con convex poliedral. Regiunea inițială se obține prin secționarea lui cu planul în care suma ponderilor este unu. Originea este inclusă matematic în con, dar nu reprezintă o distribuție normalizabilă.

Un exemplu calculat

Avem două stări posibile și trei proceduri. Prima are utilitățile (1; 0), a doua (0; 1), a treia (0,6; 0,6). Valorile sunt ipotetice și includ toate costurile presupuse. Notăm cu p probabilitatea primei stări.

Prima procedură are valoarea p, a doua 1 − p, iar a treia 0,6. Prima este optimă pentru p peste 0,6; a doua pentru p sub 0,4; a treia pentru p între 0,4 și 0,6. La frontiere există egalitate între opțiuni.

În coordonate nenormalizate, a treia procedură este preferată când fiecare dintre cele două ponderi este cel puțin două treimi din cealaltă. Obținem un sector în primul cadran. Conul descrie o familie de stări informaționale în care aceeași procedură este rațional de selectat.

Aceasta oferă un sens concret intuiției geometrice: unele „regiuni ale înțelegerii" pot fi regiuni de aplicabilitate decizională. Nu rezultă că orice concept, propoziție sau memorie este un con.

Conuri de implicare și alte semnificații

Ganea și colaboratorii folosesc conuri de implicare, inclusiv în geometrie hiperbolică, pentru reprezentarea relațiilor ierarhice [Ganea2018]. Acolo motivul construcției este păstrarea unei ordini parțiale. În derivarea noastră, motivul este comparația utilităților așteptate.

Aceste două tipuri de conuri pot fi utile într-un sistem MRP, dar nu sunt interschimbabile. Un concept subordonat într-o taxonomie nu este același lucru cu o procedură optimă într-o regiune a credințelor. Folosirea aceluiași desen nu demonstrează identitatea mecanismelor.

Ce se întâmplă cu inelele și „găurile"

Considerăm regiunea dintre două cercuri: punctele pentru care $1 < x^2 + y^2 < 4$. Ea este neconvexă. Interpretorul care calculează $r = x^2 + y^2$ transformă întrebarea „punctul aparține regiunii?" într-o verificare a intervalului $1 < r < 4$.

Transformarea pierde însă unghiul. Puncte diferite sunt comprimate în aceeași valoare. Nu am eliminat gratuit complexitatea geometrică printr-o schimbare inversabilă de co-

ordonate; am aruncat informația care nu era necesară întrebării radiale. Pentru o întrebare despre direcție, noua reprezentare ar fi insuficientă.

Găurile și conectivitatea au un statut diferit de convexitate atunci când permitem numai transformări continue cu inversă continuă. Dar interpretoarele MRP pot fi discontinue și pot comprima informația. De aceea, intuițiile despre inele sau toruri trebuie legate de o clasă explicită de transformări admise.

Ipoteza testabilă

Nu aș formula „cunoașterea are forma unor conuri”. Aș formula: pentru această familie de sarcini, o reprezentare prin anumite regiuni geometrice păstrează distincțiile relevante și permite selecția sau transferul mai eficient decât alternativele, la același buget.

Testul ar compara erorile de aplicabilitate, costul căutării, memoria și transferul. O vizualizare frumoasă nu este o dovadă. Dacă un tabel simplu obține aceleași rezultate cu un cost mai mic, ipoteza geometrică nu a produs încă valoare inginerească.

CAPITOLUL 15

Un atlas de interpretoare

De ce nu un singur spațiu

Aceeași situație poate cere păstrarea ordinii temporale, a identității participanților, a unei relații numerice sau a proveniență unei afirmații. Un spațiu unic poate codifica toate acestea în principiu, dar accesul la fiecare proprietate poate rămâne costisitor sau dificil de verificat.

Propunerea unui atlas este mai modestă: mai multe reprezentări locale, fiecare cu operații bine definite și cu un domeniu de utilizare. Termenul „atlas” este o analogie de organizare. Nu presupune automat o varietate diferențiabilă, coordonate netede sau transformări inversabile între toate reprezentările.

Un interpretor temporal poate răspunde despre ultima stare. Un interpretor relațional poate răspunde despre dependențe. Un interpretor documentar poate păstra cine susține fiecare afirmație. Ele pot descrie același episod fără a produce același tip de rezultat.

Traduceri cu contract

O traducere dintre două reprezentări trebuie să declare ce păstrează. Dacă dintr-un jurnal construim o stare curentă, putem conserva răspunsul la „ce este activ acum?”, dar nu neapărat răspunsul la „de câte ori s-a schimbat?”. O traducere utilă nu trebuie să fie complet reversibilă; trebuie să fie suficientă pentru scopul declarat.

Într-un atlas, legăturile ar fi adaptoare executabile. Fiecare adaptor are precondiții, un tip de ieșire și o afirmație despre conservarea informației. Unele conservări pot fi demonstrate. Altele pot fi numai evaluate pe teste. Statutul trebuie păstrat în metadate și în explicația rezultatului.

Un test de întoarcere, în care traducem A în B și apoi înapoi, este util pentru detectarea unor pierderi. Dar succesul pe câteva exemple nu dovedește echivalența. În plus, o traducere intenționat compresivă nu trebuie evaluată prin reconstrucția integrală, ci prin conservarea testelor relevante.

Compatibilitate locală și contradicție

Două reprezentări pot produce rezultate aparent contradictorii fără să fie logic incompatibile. Una descrie regula formală, cealaltă starea observată. O persoană poate avea dreptul de acces, dar conexiunea poate fi indisponibilă. Contradicția dispare când țintele sunt tipizate corect.

În alte cazuri, contradicția este reală: două interpretoare afirmă valori diferite despre aceeași variabilă, aceeași perioadă și aceleași ipoteze. Sistemul nu trebuie să le medieze arbitrar. Trebuie să verifice sursele, precondițiile, versiunile și dependențele.

Această muncă poate fi reprezentată ca o problemă de satisfacere a constrângerilor. Dacă un set de afirmații nu poate fi simultan adevărat în modelul declarat, agentul caută un subset conflictual și îl marchează. Un solver poate ajuta pentru limbaje potrivite, fără a rezolva automat ambiguitatea limbajului natural care a produs formulele.

Operațiile definesc coordonatele

Un atlas MRP nu ar stoca numai valori. Ar stoca și procedurile prin care ele sunt obținute. Coordonata „ultima stare validă” este un program asupra unui istoric. Coordonata „numărul de surse independente” este o procedură asupra unui graf de proveniență. Coordonata „risc estimat” este un model cu propriile date și ipoteze.

Această legătură permite verificarea unei dimensiuni noi. Putem inspecta programul, putem modifica datele de intrare și putem observa ce se schimbă. O dimensiune numerică învățată fără o interpretare simplă nu este exclusă, dar nu primește automat același tip de justificare.

Aici se găsește una dintre ideile centrale ale programului: o nouă dimensiune a înțelegerii poate fi o procedură care produce o distincție anterior inaccesibilă. Geometria rezultată depinde de procedură și de întrebările pentru care o folosim.

Cum evaluăm atlasul

Pentru fiecare sarcină putem măsura câte reprezentări au fost consultate, câte traduceri au fost executate, ce informație a fost pierdută și ce cost a avut reconcilierea. Atlasul trebuie comparat cu un spațiu unic suficient de puternic și cu o bibliotecă fixă de vederi.

Un rezultat favorabil ar fi că adaptarea locală evită reconstruirea întregii reprezentări și transferă proceduri între sarcini. Un rezultat nefavorabil ar fi că adaptoarele, selecția și verificările costă mai mult decât un model comun bine proiectat. Niciuna dintre concluzii nu trebuie anticipată prin metafora aleasă.

Atlasul nu este o obligație arhitecturală. Este o ipoteză de organizare care poate fi implementată gradual. Un prim sistem poate avea numai trei vederi și două adaptoare; important este să poată demonstra ce păstrează fiecare și când o schimbare de vedere rezolvă o dificultate reală.

PARTEA V / De la model la sistem

CAPITOLUL 16

Arhitectura unei tehnologii MRP

O arhitectură funcțională, nu o platformă presupusă

Putem descrie o tehnologie MRP fără a o confunda cu un produs deja construit. Ea ar trebui să primească o sarcină, să stabilească ce informație este necesară, să aleagă și să execute interpretoare, să verifice rezultatele și să învețe din insuficiențele identificate. Fiecare verb trebuie să corespundă unui obiect și unei tranziții observabile.

În această carte, MRP-VM desemnează o posibilă mașină de execuție pentru aceste funcții. „Mașină virtuală” nu înseamnă neapărat un nou procesor sau un nou bytecode. Înseamnă un mediu care controlează starea, apelurile, dependențele și condițiile de acceptare ale operațiilor interpretative.

O implementare minimală poate utiliza o bază de date obișnuită, un planificator de grafuri și executanți existenți. Valoarea ei nu ar proveni din numărul de tehnologii introduse, ci din posibilitatea de a lega o problemă observată de o schimbare verificabilă a repertoriului.

Obiectele persistente

Obiect	Conținut minim	Rol
Sarcină	Întrebare, intrări, criteriu de succes, buget	Definește ce trebuie rezolvat
Episod	Observații ordonate, identități, timp și surse	Permite reinterpretarea
Reprezentare	Valori, schemă, programul constructor, versiune	Păstrează distincții
Interpretor	Contract, implementare, dependențe, cost	Transformă și verifică
Evidență	Test sau demonstrație, ipoteze, rezultat, artefacte	Susține un statut precis
Incident	Predicție, observație, diferență și diagnostic	Declanșează adaptarea
Bibliotecă	Interpretoare aprobate și criterii de aplicare	Permite reutilizarea

Aceste obiecte trebuie să aibă identități stabile. Un rezultat nu poate indica doar „interpretorul temporal”, deoarece definiția lui se poate schimba. Trebuie să indice versiunea efectiv executată și intrările exacte sau identitatea lor verificabilă.

Un identificator criptografic al conținutului poate ajuta la legarea artefactelor de evidențe. El nu demonstrează adevărul rezultatului și nu înlocuiește semantica programului. Rolul său este mai restrâns: detectarea schimbării conținutului la care se referă verificarea.

Contractul unui interpretor

Contractul trebuie să precizeze ce primește interpretorul și ce poate produce. Pentru un interpretor al ultimei operații, intrarea poate fi o secvență de evenimente cu identitate și ordine totală. Precondiția poate cere ca fiecare eveniment să aibă o poziție comparabilă. Ieșirea este o valoare de stare pentru fiecare cheie.

Contractul ar declara și limitele: ignoră evenimente care nu sunt în intrare; presupune regula „ultima operație prevalează”; nu demonstrează că jurnalul este complet. Astfel, un rezultat corect despre jurnalul disponibil nu este prezentat ca dovadă despre toate evenimentele reale.

Pentru un interpretor probabilist, contractul include ținta distribuției și condițiile evaluării. Pentru un solver, include teoria și modelul verificate. Pentru un LLM, poate declara că ieșirea este o propunere textuală sau un program candidat, nu un rezultat certificat.

Controlerul și executanții

Controlerul decide ce operație urmează. Executanții pot fi interpretoare simbolice, programe, modele statistice, solvere, interogări de baze de date sau persoane solicitate să clarifice o specificație. Uniformitatea interfeței nu înseamnă uniformitatea încrederii acordate rezultatelor.

O primă versiune a controlerului poate avea reguli transparente: verifică precondițiile, alege cel mai ieftin candidat compatibil, execută, evaluează, escaladează dacă există un incident. Această versiune este un reper pentru un controler învățat ulterior. Fără reper, orice complexitate suplimentară poate părea utilă fiindcă nu are termen de comparație.

Executanții ar raporta separat timpul, consumul de memorie, operațiile externe și starea finală. Un timeout este un rezultat de execuție, nu dovada că proprietatea nu poate fi stabilită. O eroare de parsare este diferită de un contraexemplu semantic.

Registrul epistemic

Aș separa conținutul rezultatului de statutul său. „Permisiunea este activă” este conținut. „Calculat exact din jurnalul complet conform modelului M” este un statut. „Estimat din observații incomplete” este alt statut. Cele două nu trebuie să se piardă când rezultatul trece prin alte componente.

Un registru ar putea distinge observație raportată, ipoteză, rezultat derivat, verificare formală relativă la un model, validare empirică și rezultat retras. Aceste categorii nu alcătuiesc neapărat o singură scară de superioritate. O demonstrație formală despre un model greșit și o observație robustă despre sistemul real au limite diferite.

Compoziția ar trebui să păstreze dependențele relevante. Un text final bazat pe o ipoteză nu devine fapt doar pentru că ultima etapă este un generator fluent. Un verificator poate confirma forma documentului fără a confirma adevărul fiecărei afirmații.

Implementarea minimă utilă

Prima implementare nu are nevoie de un ecosistem complet. Are nevoie de o familie de sarcini finite, un nucleu de operații, câteva reprezentări inițiale, un jurnal de execuții, un generator de candidați și un evaluator separat. Acest minim permite deja testarea coliziunii, rafinării și transferului.

Extensiile ar trebui introduse numai când rezolvă o limită identificată. Dacă selecția devine costisitoare, dezvoltăm indexarea bibliotecii. Dacă evaluarea consumă prea mult, studiem strategii de alegere a testelor. Dacă nu apare transfer, investigăm inducția sau definiția sarcinilor înainte de a adăuga alte componente.

O arhitectură MRP defensibilă este una ale cărei avantaje pot fi atribuite mecanismelor sale. Nu este suficient ca toate componentele să poarte nume sugestive; trebuie să poată fi eliminate una câte una pentru a măsura ce contribuție aduc.

CAPITOLUL 17

Execuție, dovezi și revizuire

Viața unui candidat

Un interpretor nou trece prin stări distincte. Mai întâi este o propunere. Apoi este verificat structural și executat într-un mediu controlat. Ulterior poate primi o evaluare empirică sau o justificare formală pentru anumite proprietăți. Numai după îndeplinirea criteriilor stabilite este introdus în biblioteca utilizabilă.

Stare	Ce este permis	Ce nu se presupune
Propus	Inspecție, verificare de structură	Corectitudine semantică
Executabil	Testare cu resurse limitate	Generalizare
Evaluat	Utilizare în domeniul evaluat, conform politicii	Valabilitate universală
Consolidat	Reutilizare și monitorizare	Imunitate la schimbare
Suspendat	Diagnostic și reproducere	Utilizare automată curentă
Retras	Audit și reproducerea trecutului	Recomandare pentru cazuri noi

Demonstrațiile pot însoți diferite stări și pot acoperi numai o parte a contractului. „Consolidat” este o decizie de administrare a bibliotecii, nu un sinonim pentru „matematic demonstrat”. Această distincție previne transformarea unei aprobări operaționale într-o afirmație epistemică exagerată.

Un parcurs complet

Sarcina este determinarea permisiunii curente dintr-un jurnal. Reprezentarea inițială stochează numai ce tipuri de evenimente au apărut. Agentul întâlnește două istorii cu aceleași tipuri, dar cu rezultate diferite. Evaluatorul confirmă că ordinea, nu zgomotul, explică diferența.

Generatorul propune un interpretor care grupează evenimentele după persoană și resursă, selectează ultima poziție și aplică regula de stare. Verificarea structurală confirmă că tipurile se potrivesc. Testele controlează inversarea ordinii, intercalarea altor chei și repetarea operațiilor conform regulii mediului.

Un verificator suplimentar poate demonstra, pentru limbajul finit al evenimentelor și implementarea dată, că rezultatul coincide cu ultima operație relevantă. Această dovadă nu confirmă completitudinea jurnalului real; ea confirmă relația dintre intrare și ieșire în model.

După acceptare, programul este introdus în bibliotecă. La o sarcină nouă, selectorul trebuie să recunoască regula adecvată. Dacă noul protocol este cumulativ, contractul nu se aplică. Transferul corect include refuzul utilizării unei abstracții nepotrivite.

Epoci și modificări controlate

Învățarea schimbă interpretarea datelor. Dacă un interpretor este înlocuit în timpul unei execuții, rezultatul poate amesteca versiuni fără o justificare clară. O soluție este utilizarea unor epoci: fiecare execuție vede o versiune stabilă a bibliotecii și a politicii.

O modificare produce o nouă epocă. Rezultatele vechi rămân asociate configurației în care au fost obținute. Noul interpretor poate fi testat pe copii ale sarcinilor înainte de a deveni implicit. Aceasta facilitează reproducerea și compararea, fără a cere oprirea întregului sistem la fiecare învățare.

Schimbarea poate fi tranzacțională: ori sunt actualizate coerent interpretorul, schema reprezentării și dependențele afectate, ori versiunea veche rămâne activă. Conceptul de tranzacție este folosit aici ca disciplină de consistență, nu ca promisiune că orice efect extern poate fi anulat.

Efecte externe și reversibilitate

Un interpretor care citește date diferă de unul care modifică o bază de date sau trimite o comandă. Propunerile de acțiune trebuie separate de execuțiile efective. Repetarea unei verificări nu trebuie să repete involuntar un efect ireversibil.

Pentru prototipul de cercetare, putem limita operațiile la un mediu simulat sau la copii fără efect extern. Când trecem la aplicații reale, trebuie definite permisiunile, idempotența operațiilor, aprobările și evidența efectelor. O operație idempotentă are același efect asupra stării când este repetată ca atunci când este executată o singură dată. Aceste cerințe nu sunt un adaos filosofic; ele determină ce experimente sunt reproductibile.

Un sandbox limitează consecințele unor programe defecte, dar nu demonstrează că rezultatele lor sunt corecte. Invers, o dovadă despre o funcție pură nu autorizează orice utilizare a ei într-un sistem cu efecte. Cele două probleme trebuie tratate separat.

Recalcularea selectivă

Dacă o sursă este corectată sau un interpretor este retras, nu este necesar să recalculăm totul. Putem urmări graful dependențelor și marca rezultatele afectate. Această optimizare presupune însă că dependențele sunt înregistrate suficient de complet.

Un rezultat care citește indirect o configurație globală, fără a o declara, creează o dependență ascunsă. Reutilizarea lui din memorie poate fi greșită după schimbarea configurației. De aceea, identitatea unei execuții include nu numai intrarea vizibilă, ci și parametrii și versiunile relevante ale contextului.

Politica de recalculare poate începe cu o supraaproximare: invalidează toate rezultatele posibil afectate. Ulterior se poate rafina pentru a evita munca inutilă. Aceeași logică a preciziei și costului reapare la nivelul infrastructurii.

Ce explică o urmă de execuție

O urmă bună arată ce operații au fost executate, cu ce intrări și ce verificări. Ea poate explica procesul care a produs rezultatul. Nu oferă automat explicația cauzală a fenomenului din lume și nici justificarea valorică a obiectivului ales.

MRP ar trebui să livreze aceste niveluri separat. Pentru un rezultat științific, avem nevoie atât de reproducerea calculului, cât și de adecvarea modelului și a datelor. Pentru o deci-

zie organizațională, avem nevoie și de autoritatea și legitimitatea criteriilor utilizate. Transparența tehnică este valoroasă tocmai când nu se prezintă drept soluție pentru toate aceste întrebări.

CAPITOLUL 18

MRP-VM și SOP Lang

Teoria nu depinde de o sintaxă

MRP descrie organizarea interpretărilor și a învățării lor. SOP Lang poate fi o opțiune pentru reprezentarea compozițiilor executabile, dar teoria nu trebuie să depindă de o anumită extensie de fișier sau convenție de numire. Alte limbaje pot implementa același experiment.

Această separare permite două evaluări. Putem testa dacă mecanismul MRP produce transfer și eficiență. Separat, putem testa dacă reprezentarea prin circuite SOP facilitează compoziția, urmărirea provenienței și verificarea. Un eșec al sintaxei sau al parserului nu invalidează automat ipoteza generală.

Circuitul ca unitate de compoziție

Într-o implementare posibilă, un circuit este un graf de operații. Nodurile execută transformări; legăturile transportă valori și dependențe. Intrările și ieșirile sunt declarate. O operație poate apela o primitivă sau un alt circuit, iar compoziția produce o procedură mai complexă.

Pentru a simplifica analiza, rezultatele intermediare pot avea identități unice în interiorul unei execuții. În loc să modificăm tacit o valoare, producem una nouă și înregistrăm dependența. Această disciplină de atribuire unică ajută la urmărirea provenienței, fără a exclude existența unei stări persistente controlate.

Un graf fără cicluri poate fi executat în ordinea dependențelor. Procesele recursive sau iterative au nevoie de un mecanism suplimentar: o buclă explicită, un punct fix sau deschiderea unei noi etape. Simpla desenare a unui graf nu rezolvă oprirea și nici gestionarea efectelor.

Un nucleu mic, o bibliotecă extensibilă

Nucleul ar implementa operații care au nevoie de suport direct: accesul la date, verificarea tipurilor, anumite calcule și comunicarea cu executanții. Majoritatea compozițiilor de nivel superior ar putea fi circuite declarative. Astfel, un interpretor nou ar deveni un obiect inspectabil al bibliotecii, nu o modificare opacă a platformei.

Distincția „primitivă versus circuit” trebuie să fie vizibilă în documentația sistemului. Altfel, se poate pretinde că agentul a învățat o capacitate care era, de fapt, ascunsă într-o operație complexă furnizată manual. Un evaluator trebuie să poată vedea exact ce a fost compilat, învățat și apelat.

Organizarea fișierelor poate defini spațiile de nume ale comenzilor, dar această convenție nu este o contribuție cognitivă. Ea devine utilă dacă simplifică descoperirea, versionarea și reutilizarea componentelor. Evaluarea MRP trebuie să rămână centrată pe capacități, nu pe estetica organizării.

Limbaj controlat și reprezentare intermediară

O interfață în limbaj controlat poate face circuitele mai ușor de citit. Pentru un fragment riguros definit, putem cere ca reprezentarea textuală și reprezentarea intermediară să fie convertibile fără pierderi semantice în acel fragment. Aceasta este o cerință verificabilă de proiectare.

Nu putem extinde aceeași promisiune la orice propoziție din limbajul natural. Ambiguitatea, presupozițiile și contextul pot permite mai multe interpretări. Traducătorul trebuie să păstreze alternativele sau să solicite clarificări, nu să declare arbitrar că a extras „sensul exact”.

O buclă de verificare poate compara efectele circuitului cu exemple pozitive și negative furnizate de utilizator. Aceste exemple nu rezolvă toate ambiguitățile, dar fac traducerea contestabilă. Sensul stabilizat devine contractul acceptat pentru operația respectivă.

Rolul modelelor lingvistice

Un LLM poate traduce o cerere într-o propunere de circuit, poate genera teste și poate sugera o distincție nouă. Poate fi folosit și ca executant probabilist atunci când nu există o procedură simbolică potrivită. Arhitectura trebuie să permită aceste roluri fără a confunda propunerea cu verificarea.

Dacă același model generează programul și toate testele, erorile sale pot fi corelate. Sunt necesare evaluatoare independente, teste generate din specificații sau intervenții umane pentru proprietățile importante. Independența nu înseamnă neapărat un alt brand de model; înseamnă o sursă de constrângeri care nu repetă aceeași presupunere neverificată.

La început, un sistem hibrid poate fi foarte util chiar dacă majoritatea ideilor de programe provin dintr-un LLM. Dar raportul experimental trebuie să spună acest lucru. Contribuția MRP ar putea fi verificarea, consolidarea și reutilizarea, nu generarea inițială a tuturor conceptelor.

Un exemplu de compoziție fără sintaxă artificială

Pentru o sinteză documentară, un circuit poate primi un document și o familie de întrebări. O etapă identifică fragmente candidate. Alta separă afirmațiile autorului de citări și ipoteze. O a treia construiește răspunsuri cu referințe la fragmente. Un verificator controlează dacă rezumatul păstrează excepțiile cerute.

Dacă verificatorul găsește o propoziție în care o ipoteză a devenit fapt, incidentul nu cere neapărat un nou generator de text. Poate cere un interpretor care păstrează modalitatea afirmațiilor și o regulă de compoziție care interzice pierderea ei. Această nouă operație poate fi reutilizată în alte documente.

Circuitul face vizibilă distincția; MRP decide când trebuie construită și dacă merită consolidată. Aceasta este relația funcțională dintre cele două niveluri. Niciunul nu trebuie prezentat ca o soluție deja validată pentru întregul limbaj natural.

Verificare software și protocoale

De ce este un prim teren potrivit

Software-ul oferă situații în care putem controla intrările, reproduce execuțiile și formula proprietăți precise. Nu toate sistemele software sunt simplu de modelat, dar o familie delimitată de protocoale poate separa dificultatea reprezentării de ambiguitatea limbajului natural. De aceea, aceasta este prima aplicație pe care aș prioritiza-o.

Problema comercială sau inginerească ar fi reducerea costului analizelor repetate, fără a pierde proprietățile cerute. Un MRP experimental ar începe cu vederi simple asupra programului sau jurnalului și ar activa reprezentări mai precise numai acolo unde apar insuficiențe demonstrabile.

Nu este necesar ca sistemul să concureze imediat cu toate instrumentele de verificare existente. O primă contribuție poate fi selecția mai bună între analize cunoscute sau învățarea unor prelucrări care fac o proprietate accesibilă unui analizor existent.

Dosar de utilizare: protocolul de rezervare

Considerăm un serviciu de rezervări cu cerere, confirmare și anulare. Intrarea agentului este o specificație executabilă a versiunii protocolului și jurnale etichetate. Întrebarea este dacă resursa este rezervată într-un moment dat și ce continuări pot schimba starea.

Un interpretor grosier păstrează numai existența unei confirmări. El eșuează după anulare. Un al doilea păstrează ultima operație, dar poate eșua dacă protocolul distinge anularea unei cereri de anularea unei confirmări. Distincția necesară depinde de regulile exacte, nu de numele operațiilor.

Agentul poate învăța un model de stare cu precondiții. O confirmare este valabilă numai după o cerere activă, iar anumite operații sunt ignorate sau produc eroare. Testele contrastive trebuie să includă operații invalide, identități diferite și istorii mai lungi decât cele folosite la inducție.

Pentru un protocol finit, evaluatorul poate explora toate stările accesibile din model. Dacă intrările și stările sunt nelimitate, dovada exhaustivă trebuie înlocuită sau completată prin argumente inductive și teste. Mărimea domeniului verificat trebuie raportată, nu ascunsă în formula „testat complet”.

Dosar de utilizare: alegerea domeniului abstract

În analiza numerică a unui program, agentul începe cu intervale. O proprietate nu poate fi demonstrată deoarece analiza pierde egalitatea dintre două variabile. Un diagnostic identifică relația necesară, iar un interpretor relațional este activat numai pentru porțiunea relevantă.

Aici există două niveluri de învățare. Primul învață să aleagă între domenii deja implementate. Al doilea propune predicate sau transformări noi, verificate înainte de utilizare. Primul este mai simplu și constituie un reper important pentru al doilea.

Dacă generatorul propune un invariant, verificatorul trebuie să controleze inițializarea, conservarea prin tranziții și faptul că invariantul implică proprietatea cerută. Nu este suficient ca invariantul să fie adevărat în traseele observate. Pentru un set finit de trasee, multe afirmații false în general pot părea invarianti.

Interpretarea abstractă și CEGAR furnizează antecedentele tehnice pentru această disciplină [Cousot1977; Clarke2000]. Contribuția MRP ar trebui căutată în identificarea și reutilizarea distincțiilor, nu în redenumirea acestor metode.

Dosar de utilizare: migrarea unui API

O nouă versiune de serviciu poate schimba o condiție de validitate sau o regulă de prioritate. Într-un experiment controlat, agentul primește exemple din ambele versiuni și trebuie să distingă schimbarea regulii de o defecțiune accidentală.

Biblioteca veche poate rămâne utilă pentru părțile neschimbate. Agentul ar trebui să creeze un nou contract sau o nouă versiune, nu să suprascrie trecutul. Rezultatele anterioare trebuie să rămână reproductibile în raport cu versiunea lor.

Valoarea ar proveni din reducerea muncii de reanalizare: reutilizăm interpretoare compatibile, reconstruim numai distincțiile afectate și generăm teste care separă cele două versiuni. Un criteriu de succes este identificarea schimbării reale fără o creștere excesivă a alarmelor false.

Ce măsurăm

Măsurile principale ar fi proprietățile demonstrate corect, erorile ratate, alarmele false, timpul total, memoria și costul construirii noilor interpretoare. Timpul până la primul contraexemplu valid poate fi important, dar nu trebuie folosit singur: un sistem poate găsi repede un exemplu și poate rata alte clase de erori.

Comparațiile ar include o analiză unică mai precisă, o combinație fixă de analizoare și un selector fără inducție. Pentru a izola reutilizarea, sarcinile de evaluare trebuie să conțină structuri noi, nu copii ale acelorași programe cu variabile redenumite.

Un rezultat negativ informativ ar fi că selecția dintr-o bibliotecă fixă produce tot avantajul, iar inducția nu adaugă nimic după contabilizarea costului. Acest rezultat ar recomanda o tehnologie mai simplă. MRP ar rămâne un cadru de analiză util, dar ipoteza bibliotecii adaptive ar fi slăbită în domeniul testat.

Condiții de adoptare

Aș considera aplicația promițătoare atunci când există multe sarcini înrudite, specificații suficient de clare și un verificator mai fiabil decât generatorul. Nu aș începe cu sisteme în care ținta se schimbă permanent, observabilitatea este foarte slabă și evaluarea unei erori costă mai mult decât analiza manuală.

Prima integrare ar trebui să fie consultativă: sugerează analize, produce teste și explică ce relație lipsește. Abia după o evaluare independentă ar putea automatiza decizii cu efecte. Utilitatea imediată poate fi accelerarea muncii unui inginer, fără pretenția unui verificator autonom universal.

CAPITOLUL 20

Documente, sinteză și proceduri

Rezumatul ca reprezentare cu pierderi

Un rezumat comprimă. Întrebarea nu este dacă pierde informație, ci dacă pierde informația necesară utilizării sale. Un rezumat pentru orientare generală și unul pentru aplicarea unei proceduri au criterii diferite. Aceeași reducere poate fi acceptabilă în primul caz și periculoasă în al doilea.

MRP ar reprezenta această cerință printr-o familie de întrebări și transformări relevante. Trebuie păstrate entitățile, condițiile, excepțiile, temporalitatea și statutul afirmațiilor în măsura în care influențează răspunsurile. Nu presupunem că această familie poate acoperi toate întrebările viitoare.

Un rezultat documentar ar include rezumatul, legăturile către fragmentele-sursă și un raport asupra verificărilor efectuate. Aceasta ar permite utilizatorului să distingă „nu am găsit o excepție” de „am verificat toate secțiunile relevante ale acestui document conform procedurii definite”.

Dosar de utilizare: o procedură cu excepții

Imaginăm o procedură internă: cererile se aprobă automat până la o anumită valoare, cu excepția celor care implică o resursă critică; excepțiile cer o aprobare suplimentară. Textul este un exemplu inventat, nu o descriere juridică sau organizațională reală.

Un rezumat care păstrează numai pragul poate produce răspunsuri greșite. Reprezentarea sa confundă două cereri cu aceeași valoare, dar cu resurse diferite. Contraexemplul indică o distincție condițională, nu neapărat o problemă de fluență.

Un interpretor nou poate extrage regula generală, excepția și relația de prioritate. Altul poate verifica dacă o propoziție din rezumat omite o condiție necesară. Pentru un fragment controlat, regulile pot deveni predicate executabile. Pentru texte ambigue, sistemul trebuie să păstreze interpretări candidate și să marcheze întrebarea nerezolvată.

Testele ar varia valoarea cererii, tipul resursei și prezența aprobării. Cazurile de evaluare ar include excepții formulate altfel și combinații nevăzute, nu doar copierea expresiei „cu excepția”. Astfel separăm recunoașterea lexicală de păstrarea relației logice.

Dosar de utilizare: afirmații și atribuiri

Un raport spune că o echipă susține ipoteza H, dar că rezultatele nu o confirmă încă. Un rezumat defect poate afirma direct H. Informația pierdută nu este o dată numerică, ci modalitatea și atribuirea: cine susține ce și cu ce statut.

Un circuit documentar ar putea construi înregistrări de forma: conținutul afirmației, autorul atribuit, caracterul ipotetic sau asertat, fragmentul-sursă și relația cu evidențele. Reprezentarea nu garantează extragerea corectă, dar permite testarea separată a acestor dimensiuni.

Un verificator ar compara afirmațiile rezumatului cu înregistrările acceptate. Dacă statutul este mai puternic decât cel al sursei, ar semnala o promovare nejustificată. Acest tip de distincție este reutilizabil între documente și poate constitui o țintă bună pentru învățarea bibliotecii.

Două verigi care nu trebuie confundate

Prima verigă este traducerea textului în reprezentare. A doua este calculul asupra reprezentării. Un solver poate demonstra impecabil că o anumită concluzie decurge din regulile extrase, în timp ce extragerea însăși este greșită. Evaluarea trebuie să măsoare ambele verigi.

Pentru un experiment inițial, putem genera documente din structuri cunoscute și varia exprimarea lor. Aceasta furnizează o referință exactă pentru relațiile urmărite. Ulterior trebuie folosite documente autentice, cu adnotare independentă și cu posibilitatea de dezacord între evaluatori.

Datele sintetice sunt utile pentru izolarea mecanismului, dar pot fi prea apropiate de limbajul generatorului. Succesul pe ele nu dovedește automat utilitatea pe documente reale. Cele două etape trebuie raportate separat.

Criterii de calitate

Acuratețea răspunsurilor la întrebări relevante este un criteriu, nu singurul. Mai contează acoperirea, păstrarea excepțiilor, fidelitatea atribuirii, localizarea surselor și rata de

abținere justificată. Un rezumat foarte scurt poate răspunde corect la trei întrebări cunoscute și poate deveni inutil pentru restul sarcinii.

O parte a întrebărilor de evaluare trebuie să fie ascunsă generatorului. Trebuie inclusă și evaluarea umană a utilității, fără a o confunda cu preferința pentru stil. Cititorul poate prefera un text fluent care omite tocmai avertismentul important.

Costul trebuie comparat cu o soluție de referință rezonabilă: un LLM cu acces la surse, căutare și verificări similare. Dacă MRP primește un verificator și o memorie suplimentară pe care alternativa nu le primește, experimentul nu izolează arhitectura.

Utilitate și limite

Aplicația ar putea fi valoroasă pentru manuale tehnice, documentații de produse, proceduri interne și sinteze de cercetare. Bibliotecile de interpretoare ar păstra distincții recurente: condiție versus concluzie, ipoteză versus fapt raportat, stare curentă versus istorie, sursă primară versus citare indirectă.

În domenii cu consecințe juridice sau medicale, această arhitectură ar trebui studiată ca suport documentar cu control de specialitate, nu prezentată ca autoritate autonomă. O dovadă despre structura unui rezumat nu este o validare profesională a recomandărilor sale.

Avantajul urmărit este reducerea erorilor identificabile și a costului de verificare pe documente repetate. Nu este promisiunea că orice sens natural poate fi complet formalizat. O tehnologie utilă poate avea limite explicite și poate rezolva foarte bine numai anumite tipuri de distincții.

CAPITOLUL 21

Cercetare și descoperire

Trei activități diferite

Un agent de cercetare poate genera ipoteze, poate alege experimente și poate evalua rezultate. Aceste activități nu sunt același lucru. O ipoteză interesantă poate fi falsă; un experiment bine ales poate produce un rezultat negativ; o analiză corectă poate să nu conducă la o descoperire nouă.

MRP ar putea organiza relația dintre ele prin interpretoare cu roluri diferite. Unele construiesc modele candidate. Altele calculează predicții. Altele identifică teste care separă modelele sau verifică dacă o concluzie depășește datele. Succesul fiecărui rol trebuie evaluat potrivit țintei sale.

Nu aș defini noutatea prin surpriza internă a unui model. O afirmație poate fi rară în reprezentarea agentului și bine cunoscută în literatură. Invers, o idee nouă poate fi exprimată în termeni foarte obișnuiți. Evaluarea priorității științifice cere surse externe și o delimitare a contribuției.

Dosar de utilizare: două mecanisme compatibile cu datele

Presupunem un sistem simulat în care două modele explică aceleași observații pasive. În primul, X și Y au o cauză comună. În al doilea, X influențează Y. Exemplul din capitolul 7 arată că observarea lui X și Y poate să nu separe aceste mecanisme, în timp ce o intervenție poate să o facă.

Un interpretor predictiv identifică acordul modelelor pe datele existente. Un interpretor de intervenții calculează ce experimente au rezultate diferite. Meta-controlerul compară costul experimentelor cu valoarea separării. După observație, biblioteca de modele este actualizată.

Într-o microlume, evaluatorul cunoaște mecanismul real. Putem măsura câte experimente sunt necesare pentru identificare și ce cost total au. Într-un laborator real, ipotezele despre intervenții, instrumente și erori devin parte din problemă. Distincția observație–intervenție este fundamentală, dar nu înlocuiește proiectarea experimentului [Pearl2018].

Învățarea unei proceduri de descoperire

O contribuție mai interesantă decât rezolvarea unui singur caz ar fi învățarea unei proceduri reutilizabile. De exemplu, atunci când două modele diferă numai printr-o relație, agentul caută o intervenție care blochează explicația alternativă. Procedura trebuie formulată în condiții în care acest raționament este valid.

Nu putem transfera mecanic aceeași schemă la orice sistem. Intervenția poate schimba alte variabile, poate fi imposibilă sau poate avea efecte întârziate. Un interpretor reutilizabil ar trebui să solicite aceste condiții, nu să le presupună.

Prin urmare, biblioteca de cercetare ar stoca atât șabloane de experiment, cât și precondițiile lor. Învățarea nu ar însemna numai „ce experiment a mers”, ci „în ce clasă de situații acel experiment identifică diferența relevantă”.

Compresie, explicație și descoperire

Un model mai scurt poate comprima observațiile. Aceasta este o proprietate utilă, dar insuficientă pentru o explicație cauzală. O formulă interpolatoare poate prezice exemplele existente fără a surprinde mecanismul intervențiilor. Un concept reutilizabil trebuie evaluat pe consecințe noi.

Aș cere trei teste distincte pentru o ipoteză: potrivire cu datele cunoscute, predicții în condiții nevăzute și, acolo unde sensul cauzal este revendicat, predicții despre intervenții adecvate. Absența celui de-al treilea test nu invalidează un model predictiv, dar limitează interpretarea sa.

MRP poate face aceste limite vizibile în registrul epistemic. Un rezultat poate fi excelent ca predictor și încă nesusținut ca explicație mecanistică. Această separare este utilă pentru cercetători tocmai fiindcă evită trecerea tacită de la performanță la ontologie.

Dosar de utilizare: revizuirea unei literaturi

Agentul primește un corpus delimitat de articole și o întrebare de cercetare. Un interpretor extrage afirmațiile relevante, altul identifică metodele și condițiile studiilor, iar altul separă confirmările independente de citările aceleiași dovezi.

Un incident apare când două rezultate par incompatibile. Agentul poate descoperi că populațiile, măsurătorile sau definițiile diferă. Distincția nouă nu rezolvă automat disputa, dar poate reformula întrebarea astfel încât conflictul să devină investigabil.

Evaluarea ar măsura corectitudinea atribuirilor, identificarea condițiilor și utilitatea întrebărilor propuse pentru experți. Nu aș folosi numărul de „idei noi” declarate de agent ca metrică principală. Este prea ușor de crescut prin reformulări sau prin ipoteze neverificabile.

Relația cu cercetătorul uman

Cercetătorul stabilește problemele importante, limitele etice și criteriile de relevanță. Agentul poate ajuta la identificarea unor distincții ignorate și la explorarea consecințelor lor. Aceasta nu presupune o separare rigidă între creativitate umană și calcul automat; presupune responsabilități și evidențe explicite.

Un prim sistem ar trebui să producă dosare de ipoteză: model, presupuneri, date folosite, predicții discriminante, experimente propuse și motivele pentru care rezultatul ar conta. Valoarea sa poate fi evaluată înainte de existența unei descoperiri spectaculoase, prin timpul economisit și erorile metodologice detectate.

Aplicația este ambițioasă, dar poate fi construită gradual. Primul obiect realist este un agent care face cercetarea mai verificabilă și selectează mai bine experimentele într-un domeniu restrâns. Nu este o mașină care garantează noutatea sau înlocuiește validarea comunității științifice.

CAPITOLUL 22

Educație și agenți interactivi

Educația ca diagnostic al distincțiilor

Un elev sau un student poate aplica o regulă pe exemple obișnuite, dar poate confunda două situații care cer tratamente diferite. Un sistem educațional MRP ar încerca să identifice această coliziune în răspunsuri și să propună exemple contrastive. Aceasta este o direcție de proiectare, nu o concluzie empirică despre eficiența pedagogică.

De exemplu, un student poate crede că ordinea nu contează în orice compoziție. Sistemul îi oferă operații numerice comutative și operații de stare necomutative. Întrebarea nu este dacă studentul memorează o excepție, ci dacă poate recunoaște ce proprietate a operațiilor justifică sau interzice schimbarea ordinii.

Agentul ar păstra un model explicit al ipotezelor despre dificultatea studentului. Un răspuns greșit poate proveni din neînțelegerea conceptului, dintr-o eroare aritmetică sau dintr-o formulare ambiguă. Nu trebuie transformat automat într-o etichetă permanentă despre capacitatea persoanei.

Dosar de utilizare: interpretarea unei condiții

Un cursant confundă „dacă A, atunci B” cu „dacă B, atunci A”. Sistemul generează situații în care implicația inițială este adevărată, dar inversa este falsă. Apoi schimbă domeniul exemplelor, pentru a vedea dacă distincția este transferată.

Interpretorul pedagogic propune o explicație și un set de întrebări. Un evaluator separat măsoară progresul pe probleme nevăzute. Memoria ar trebui să păstreze ce explicații au fost încercate și în ce condiții au ajutat, nu să reducă persoana la un scor unic.

Valoarea MRP ar fi în selecția unor intervenții pedagogice adecvate tipului de coliziune. Dar sistemul trebuie comparat cu metode de predare bine proiectate și cu o versiune adaptivă fără inducția unor noi interpretoare. Utilitatea nu poate fi dedusă numai din eleganța modelului.

Ce nu trebuie optimizat

Maximizarea răspunsurilor corecte imediat după explicație poate favoriza memorarea. Maximizarea timpului petrecut în aplicație poate favoriza dependența de ajutor. Un experiment educațional ar trebui să urmărească retenția, transferul și autonomia, în condițiile aprobate pentru participanți.

Evaluarea pe oameni cere un protocol adecvat și protejarea datelor. Înaintea ei, mecanismul poate fi testat cu agenți simulați care au erori cunoscute. Aceste rezultate ar valida diagnosticul în model, nu ar demonstra efecte pedagogice reale.

Arhitectura trebuie să permită profesorului sau cursantului să conteste diagnosticul. O explicație tehnică despre „distincția lipsă” nu justifică reducerea unei dificultăți umane complexe la o singură reprezentare internă presupusă.

Agenți care observă și acționează

Un agent interactiv poate alege să privească, să măsoare, să întrebe sau să execute o operație. În aceste situații, interpretoarele nu primesc numai simboluri textuale, ci și imagini,

valori de senzori sau rezultate de instrumente. Modelul MRP nu exclude aceste intrări, dar trebuie să declare ce prelucrări sunt furnizate.

Un exemplu simplu este un dispozitiv simulat care poate fi „pornit”, „defect” sau „în așteptare”. Același indicator luminos poate apărea în mai multe stări. Un interpretor bazat numai pe imagine nu poate separa situațiile. O măsurătoare suplimentară sau o comandă de test poate produce distincția necesară.

Aici învățarea unui interpretor și alegerea unei observații devin complementare. Noul program poate spune cum se combină indicatorul cu răspunsul la test. Dar fără efectuarea testului, programul nu obține informație despre starea curentă. Aceasta menține separarea dintre calcul și achiziția de date.

Dosar de utilizare: asistent pentru fluxuri de lucru

Un asistent poate gestiona pași într-un proces administrativ sau tehnic: cerere, verificare, aprobare, execuție. La început, el primește stări explicite și operații simulate. Scopul este să aleagă următoarea acțiune permisă și să detecteze ce informație lipsește.

O coliziune apare dacă două cereri au aceleași câmpuri vizibile, dar una a primit o aprobare revocată ulterior. Agentul trebuie să păstreze istoricul relevant sau să consulte sursa autoritativă. O altă coliziune poate proveni din diferența dintre „aprobat” și „executat”.

Un sistem util ar învăța verificări reutilizabile pentru aceste distincții, fără a executa automat acțiuni în afara mandatului. Evaluarea ar măsura pașii incorecți, cererile inutile de informație, abținerile și costul total. Viteza nu este suficientă dacă agentul trece peste condiții obligatorii.

Unde se oprește analogia cu înțelegerea

Un agent poate învăța să distingă stările unui dispozitiv și totuși să nu înțeleagă importanța socială a operației pe care o execută. El poate avea o reprezentare suficientă pentru testele tehnice, dar insuficientă pentru consecințele asupra persoanelor.

Aceasta nu face modelul inutil. Arată că familia de teste și obiective trebuie extinsă cu grijă și că unele judecăți rămân în responsabilitatea oamenilor. Înțelegerea operațională este întotdeauna însoțită de o delimitare: în raport cu ce sarcini, ce observații și ce consecințe este suficientă?

Aplicațiile interactive sunt promițătoare atunci când putem observa rezultatele, putem limita efectele și putem controla costul experimentelor. În contexte opace sau ireversibile, aceeași arhitectură trebuie să adopte un rol mai prudent, orientat spre recomandare și clarificare.

CAPITOLUL 23

Programul experimental

Afirmația principală și domeniul ei

O ipoteză empirică utilă trebuie să numească sarcinile, sistemele comparate și criteriul de succes. Pentru prima etapă, aș formula astfel: într-o familie de procese discrete cu structură reutilizabilă, un agent care diagnostichează coliziuni, induce distincții executabile și le consolidează obține un compromis eroare–cost mai bun decât alternativele cu acces comparabil la informație și instrumente.

Această afirmație poate fi falsă. Familiile alese pot să nu ofere suficient transfer; inducția poate fi prea scumpă; selectorul poate greși; un model unic poate învăța aceeași structură mai eficient. Protocolul trebuie să poată produce și aceste concluzii.

Nu ar trebui început cu obiectivul vag „demonstrarea înțelegerii”. Ar trebui început cu proprietăți observabile, a căror combinație susține sau slăbește interpretarea propusă. Denumirea MRP nu trebuie să devină un substitut pentru măsurare.

Etapele și pragurile de trecere

Etapă	Obiect de studiu	Condiție pentru continuare
1. Coliziuni exacte	Istorii finite, etichete deterministe	Diagnosticul separă calculul de pierderea reprezentării

2. Inducție controlată	Compoziții din primitive declarate	Rafinările rezolvă cazuri structurale nevăzute
3. Bibliotecă	Serii de sarcini înrudite	Reutilizarea amortizează inducția și selecția
4. Incertitudine	Observații stocastice și date incomplete	Estimările și abținerile sunt evaluate corect
5. Texte controlate	Documente generate din structuri cunoscute	Sunt păstrate condițiile, atribuirea și temporalitatea
6. Aplicație pilot	Date autentice și utilizatori ai domeniului	Avantajul supraviețuiește costurilor și erorilor reale

Pragurile numerice trebuie stabilite înaintea evaluării finale, în funcție de aplicație și de un studiu pilot. Nu aș inventa aici o acuratețe obligatorie sau un procent universal de economie. Important este ca criteriul să fie fixat înainte de compararea decisivă, nu ales după rezultate.

Sistemele de referință

Primul reper este un model unic capabil, antrenat pe aceleași sarcini. Al doilea este o bibliotecă fixă cu un selector. Al treilea este un generator de programe care rezolvă fiecare sarcină separat, fără consolidare. Al patrulea este sistemul MRP complet. Se poate adăuga un reper ideal care primește reprezentarea corectă, pentru a estima cât se pierde din cauza inducției.

Reperul ideal nu este un competitor realist și trebuie etichetat ca atare. Rolul lui este diagnostic. Dacă el funcționează, dar toate variantele de inducție eșuează, problema este descoperirea reprezentării, nu imposibilitatea sarcinii în limbajul ales.

Când folosim un LLM, toate variantele relevante trebuie să primească acces comparabil la el. Trebuie raportate prompturile, apelurile, tokenurile sau alte unități de consum, plus timpul și memoria locală. Nu este corect să contabilizăm numai ultima execuție simbolică după o căutare externă costisitoare.

Ablarea mecanismelor

O ablație elimină sau înlocuiește o componentă, păstrând restul cât mai constant. Fără diagnostic, generatorul primește numai eroarea. Fără consolidare, fiecare soluție rămâne locală. Fără proveniență, agentul nu poate identifica dependențele schimbate. Fără selecție învățată, folosim o regulă simplă.

Dacă eliminarea unei componente nu schimbă rezultatele, ea nu are încă o contribuție demonstrată în experiment. Poate fi necesară în alte condiții, dar această posibilitate nu justifică revendicarea unui avantaj actual. Ablarea trebuie privită ca instrument de simplificare, nu ca obstacol retoric.

Pentru geometrie, alternativele pot fi un tabel de aplicabilitate, o reprezentare euclidiană și o reprezentare cu regiuni inspirate de conuri. Se compară păstrarea relațiilor relevante și costul, nu numai eroarea unei proiecții vizuale.

Separarea structurală a datelor

Împărțirea aleatorie a exemplelor poate lăsa aceeași structură în antrenare și testare. Un program învață atunci un șablon familiar, iar rezultatul este prezentat ca generalizare. Aș separa datele după reguli, compoziții, lungimi de dependență și combinații de condiții.

O evaluare poate reține pentru testare toate protocoalele în care o operație anulează efectul alteia numai pentru o anumită cheie. Alta poate reține combinații dintre ordonare și agregare. Astfel, transferul măsoară ceva mai puternic decât schimbarea denumirilor.

Același principiu se aplică documentelor. Fragmentele din același document sau generate din aceeași structură nu trebuie tratate ca observații complet independente. Unitatea de eșantionare poate fi documentul, protocolul sau familia, nu fiecare propoziție ori fiecare traseu aproape identic.

Incertitudine statistică

Rezultatele trebuie raportate pe mai multe sarcini și inițializări, cu variația dintre ele. Un avantaj mediu poate ascunde eșecuri grave într-o subfamilie. Este utilă raportarea distribuției costurilor și a erorilor, nu numai media.

Dacă un candidat fix nu face nicio eroare în n teste independente din aceeași distribuție, eroarea sa reală nu este demonstrată zero. O limită unilaterală de încredere de 95% este 1 − 0,05 la puterea 1/n, în modelul binomial. Anexa explică derivarea și de ce ea nu se aplică automat după selecție adaptivă pe aceleași teste.

Oprirea experimentului exact când apare un rezultat favorabil și omiterea încercărilor anterioare pot distorsiona concluziile. Protocolul trebuie să precizeze bugetul, condițiile

de oprire și modul de selecție a modelului. Cercetarea asupra validării adaptive este relevantă pentru această disciplină [Dwork2015].

Ce ar infirma ipotezele

Ipoteză	Rezultat care o slăbește direct
Distincțiile învățate sunt generale	Avantajul dispare pe compoziții nevăzute
Biblioteca aduce economie	Costul total depășește rezolvarea independentă
Diagnosticul este util	O căutare fără diagnostic obține același rezultat
Meta-controlul merită	O regulă simplă domină după includerea costului selecției
Geometria oferă structură utilă	Nu îmbunătățește aplicabilitatea sau transferul
Traducerea în circuite ajută	Erorile de traducere depășesc avantajul verificării ulterioare

Un rezultat negativ trebuie urmat de localizarea cauzei, nu de schimbarea arbitrară a definiției succesului. Putem descoperi că ideea funcționează numai pentru procese cu reguli stabile sau pentru biblioteci suficient de reutilizate. O delimitare mai îngustă, susținută de date, este un progres real.

Artefactele necesare

Un pachet de cercetare ar trebui să conțină generatorul sarcinilor, specificațiile, distribuțiile de date, primitivele disponibile, programele induse, jurnalele complete de cost, evaluatorul și rezultatele negative. Pentru un studiu cu texte, ar include sursele și regulile de adnotare în limitele permise de utilizarea lor.

Raportul final trebuie să distingă rezultatele demonstrate, cele măsurate și cele încă propuse. Această carte furnizează un program și derivări conceptuale; nu prezintă rezultate ale unui asemenea pachet experimental. Prima publicație tehnică ar trebui să se concentreze pe un mecanism restrâns și reproductibil, înaintea unei afirmații despre inteligență în general.

CAPITOLUL 24

Limite, obiecții și contribuții

„Totul este un interpretor”

Aceasta este cea mai puternică obiecție de vacuitate. Dacă orice funcție care transformă o intrare este numită interpretor, atunci orice sistem poate fi redescris în vocabularul MRP. O redescriere atât de generală poate fi filosofic interesantă, dar nu discriminează între arhitecturi.

Răspunsul este să restrângem programul empiric: interpretoare explicit reprezentate, domenii de aplicare, coliziuni observabile, mecanisme de inducție și criterii de transfer. Nu pretindem că un sistem lipsit de aceste structuri nu poate înțelege. Investigăm dacă aceste structuri produc anumite avantaje.

Partea descriptivă poate rămâne o perspectivă generală. Partea tehnică trebuie să accepte comparația cu sisteme care nu folosesc terminologia noastră. Un rezultat bun al unui model monolitic este informație relevantă, nu un motiv de a redefini retrospectiv modelul ca MRP pentru a salva ipoteza.

„Este doar semantică operațională”

Semantica operațională poate descrie execuția sistemului, inclusiv învățarea și schimbarea interpretoarelor. MRP nu dovedește că există un proces cognitiv imposibil de exprimat semantic. O asemenea afirmație ar cere un argument mult mai puternic decât cel oferit aici.

Distincția utilă este între descrierea regulilor unui sistem și problema selectării sau construirii regulilor de interpretare potrivite unei sarcini. MRP se concentrează pe a doua. Ea poate fi formalizată semantic fără să își piardă utilitatea, după cum un algoritm de învățare poate avea o semantică exactă.

Formularea „semanticile sunt pragmatici restrânse” este acceptabilă ca alegere de vocabular în acest cadru, dacă înseamnă regimuri de interpretare stabilizate. Nu este o clasificare obligatorie a tuturor teoriilor despre sens și nu trebuie folosită pentru a minimaliza contribuțiile semanticii formale.

„Este o combinație de idei cunoscute”

Obiecția este, în mare parte, corectă. Echivalențele comportamentale, abstracțiile, învățarea activă, inducția programelor și metaraționamentul au antecedente solide. O arhitectură cu mai multe module și un selector nu este nouă prin simpla existență a pluralității; amestecurile ierarhice de experți oferă un precedent clar pentru împărțirea intrărilor între componente specializate [Jordan1993].

Diferența urmărită de MRP nu este absența rutării în alte sisteme. Este trasabilitatea unei tranziții complete: o eroare este diagnosticată ca insuficiență de reprezentare, se construiește o nouă distincție executabilă, se verifică, apoi se măsoară reutilizarea ei. Această combinație poate produce o contribuție, dar noutatea exactă trebuie demonstrată prin comparație și experiment.

Direcție existentă	**Legătură cu MRP**	**Ce ar trebui adăugat sau măsurat**
Interpretare abstractă	Mai multe domenii pentru proprietăți diferite	Alegerea și inducția ghidate de incidente
Învățarea automatelor	Stări definite prin continuări distincte	Biblioteci reutilizabile între sarcini
Reprezentări predictive	Stare construită din predicții de teste	Selecție, cost și rafinare explicită
Învățarea programelor	Concepte ca abstracții executabile	Diagnosticul informației pierdute

Amestecuri de experți	Specializare și rutare	Contracte și geneza unei distincții noi
Metaraționament	Alegerea calculului sub costuri	Valoarea construirii unei biblioteci

Tabelul propune întrebări de integrare, nu afirmă că niciun sistem anterior nu a explorat aceste combinații. Bibliografia este selectivă și nu stabilește o revendicare de prioritate. Publicarea unui mecanism nou ar cere o analiză mai focalizată a literaturii imediat relevante.

„Un model mare poate face deja toate acestea”

În principiu, un model suficient de expresiv poate implementa intern proceduri similare. MRP nu are nevoie de o imposibilitate universală a modelelor monolitice pentru a fi utilă. Avantajul urmărit poate fi controlul, verificarea locală, costul sau adaptarea cu puține exemple.

Comparația relevantă este operațională: ce poate face fiecare sistem la același buget și cu aceleași informații? Un model mare poate câștiga. Un sistem hibrid poate câștiga. Rezultatul poate depinde de frecvența sarcinilor și de costul erorilor. Teoria trebuie să permită această pluralitate.

Este posibil și ca cea mai bună tehnologie să folosească un model mare pentru propuneri și interpretoare mici pentru execuții repetate. Aceasta nu este o contradicție. Arată că unitatea utilă de comparație este sistemul complet și că rolurile trebuie atribuite corect.

„Unde este înțelegerea, dincolo de succes?”

Performanța pe teste nu epuizează înțelegerea umană. Un agent poate exploata regularități accidentale sau poate avea competență foarte îngustă. MRP încearcă să reducă această problemă prin teste contrastive, transfer structural și verificarea procedurilor, dar nu oferă un criteriu definitiv al conștiinței ori al sensului trăit.

Problema ancorării simbolurilor rămâne relevantă [Harnad1990]. Legarea unei reprezentări de observații și intervenții oferă o ancorare operațională, însă nu rezolvă prin definiție toate întrebările filosofice despre semnificație. Trebuie păstrată diferența dintre o contribuție inginerească și o teorie completă a minții.

În același timp, absența unei teorii complete nu împiedică testarea unor capacități concrete. Putem demonstra că o reprezentare pierde o distincție și putem arăta că un program nou o recuperează din datele disponibile. Acesta este un progres precis, chiar dacă nu răspunde tuturor întrebărilor despre înțelegere.

Limite ale calculabilității și ale datelor

Nu există o procedură generală care să decidă toate proprietățile semantice interesante ale programelor arbitrare; computabilitatea impune limite, iar complexitatea limitează practic multe probleme decidabile [Barak]. De aceea, inducția și verificarea trebuie să lucreze în limbaje restrânse, cu aproximări sau cu posibilitatea de a nu concluziona.

Niciun limbaj mai expresiv nu poate crea informația empirică lipsă despre lumea curentă. Nicio bibliotecă finită nu garantează suficiența pentru toate întrebările viitoare. Aceste limite nu cer abandonarea programului; cer domenii explicite și criterii de extindere.

Un sistem poate avea dreptate să nu învețe un interpretor nou. Dacă sarcina este unică, informația lipsește sau costul depășește beneficiul, abținerea poate fi cea mai bună opțiune. Adaptarea permanentă nu este un scop în sine.

Ce merită păstrat

Contribuția conceptuală cea mai puternică este mutarea atenției de la reprezentări privite ca obiecte statice la distincții construite prin proceduri. Contribuția matematică a acestei formulări este modestă, dar clară: putem defini suficiența, coliziunile, distanțele și unele limite ale rafinării. Rezultatele elementare prezentate aici nu sunt revendicate ca teoreme noi în literatura de specialitate.

Contribuția inginerească posibilă este un mediu în care aceste obiecte sunt executabile, versionate și evaluate. Contribuția empirică rămâne de stabilit: dacă un asemenea mediu învață și transferă mai eficient decât alternativele bine construite.

În forma sa cea mai defensibilă, Meta-Rational Pragmatics nu afirmă că a găsit forma ultimă a cunoașterii. Propune o întrebare de lucru: ce diferență relevantă nu poate face încă agentul, ce procedură ar face-o posibilă și ce dovadă arată că merită păstrată? O tehnologie reală poate începe exact aici.

Anexe / Instrumente pentru studiu și verificare

ANEXA A

Rezultate matematice elementare

Rezultatele de mai jos sunt demonstrații didactice pentru modelul folosit în carte. Nu sunt revendicate ca teoreme originale. Ele arată exact unde apare o garanție și ce presupuneri nu pot fi eliminate. Folosim domenii finite atunci când acest lucru evită dificultăți de măsurabilitate sau de existență a unei reprezentări efective.

A1. Limita unei reprezentări care confundă etichete

Considerăm N exemple, fiecare cu o etichetă corectă, și o reprezentare R. Împărțim exemplele în clase după valoarea R. Un predictor care vede numai R trebuie să folosească aceeași distribuție de răspunsuri pentru toate exemplele unei clase.

Într-o clasă, cel mai bun răspuns determinist este eticheta cea mai frecventă. Nici randomizarea nu poate obține o acuratețe medie mai mare: media ponderată a frecvențelor etichetelor nu depășește frecvența maximă. Numărul maxim de răspunsuri corecte în clasă este deci frecvența celei mai comune etichete.

Eroare minimă empirică = 1 − (suma frecvențelor etichetelor majoritare din fiecare clasă) / N.

Pentru două istorii echiprobabile cu aceeași reprezentare și etichete opuse, rezultă eroarea minimă 0,5. Limita se aplică predictorului care nu primește alte informații. Dacă îi oferim identitatea exemplului sau istoricul original, am schimbat experimentul.

A2. Suficiența și factorizarea răspunsului

Fie f răspunsul corect la o întrebare determinată și R o reprezentare pe un domeniu finit. Există o funcție g astfel încât f(h) = g(R(h)) pentru orice h dacă și numai dacă toate istoriile cu aceeași valoare R au același răspuns f.

Sensul direct este imediat: aceeași intrare pentru g produce același rezultat. Pentru sensul invers, definim g pe fiecare valoare R prin răspunsul comun al clasei sale. Condiția garantează că definiția nu este ambiguă.

Rezultatul afirmă existența unei funcții, nu eficiența ei. Tabelul g poate fi enorm. Pentru domenii infinite, existența matematică a unei factorizări nu implică automat o procedură calculabilă sau accesibilă agentului.

A3. Rafinarea nu înrăutățește riscul optim fără costuri

Presupunem că noua reprezentare păstrează vechea reprezentare și adaugă D: Rnou(h) = (R(h), D(h)). Orice predictor vechi poate fi simulat pe noua reprezentare ignorând componenta D. Prin urmare, cel mai mic risc posibil pe noua reprezentare nu poate fi mai mare decât cel mai mic risc posibil pe cea veche.

Concluzia privește predictorul optim într-o clasă suficient de largă. Un algoritm concret antrenat pe date finite poate performa mai slab după adăugarea unei componente. Pot crește costul, dificultatea estimării sau riscul de supraînvățare. Teorema nu garantează progresul unui sistem real după fiecare rafinare.

A4. Numărul rafinărilor stricte într-un eșantion finit

O partiție a N obiecte are cel mult N clase. Dacă pornim de la K clase și fiecare rafinare strictă mărește numărul claselor cu cel puțin unu, pot exista cel mult N − K rafinări stricte, atât timp cât nu reunim ulterior clasele.

Demonstrația este o numărare. După m rafinări avem cel puțin K + m clase; deoarece K + m nu poate depăși N, rezultă m ≤ N − K. Limita nu spune cât costă găsirea fiecărei rafinări. Nu spune nici că partiția finală generalizează: separarea fiecărui exemplu poate însemna doar memorare.

A5. Pseudometrica testelor

Pentru fiecare test t, variația totală satisface TV(P1, P3) ≤ TV(P1, P2) + TV(P2, P3). Fiecare termen din dreapta este cel mult supremul corespunzător peste toate testele. Luând supremul și în stânga, obținem dT(h1, h3) ≤ dT(h1, h2) + dT(h2, h3).

Nenegativitatea și simetria se moștenesc direct. Distanța poate fi zero între istorii distincte deoarece testele nu le separă. Pe clasele de istorii cu distanță zero, funcția devine o metrică. Adăugarea unor teste nu poate micșora această distanță definită prin suprem, dar poate crește costul estimării ei.

A6. Limita de 2e pentru alegerea unei acțiuni

Presupunem un set finit de acțiuni. Valoarea estimată a fiecărei acțiuni diferă de valoarea reală cu cel mult e. Notăm cu a* o acțiune optimă în realitate și cu â o acțiune cu estimarea maximă. Simbolul â este doar un nume pentru alegerea pe baza estimării.

Valoarea reală a lui a* este cel mult estimarea sa plus e. Estimarea sa nu depășește estimarea lui â, prin definiția alegerii. Estimarea lui â este cel mult valoarea reală a lui â plus e. Combinând cele trei relații, valoarea reală a lui a* este cel mult valoarea reală a lui â plus 2e.

Rezultatul cere controlul erorii pentru toate acțiunile comparate. O eroare medie mică, fără controlul acțiunii selectate, nu oferă aceeași garanție. Pentru planuri lungi, trebuie justificat separat ce reprezintă valoarea estimată a fiecărei politici.

A7. Zero erori nu înseamnă eroare zero

Presupunem un candidat fix, teste independente și aceeași probabilitate reală r de eroare la fiecare test. Probabilitatea de a observa zero erori în n teste este (1 − r) la puterea n. Alegem limita r astfel încât această probabilitate să fie 0,05. Rezultă r = 1 − 0,05 la puterea 1/n.

Pentru n = 100, limita este aproximativ 0,0295, adică 2,95%. Observarea a zero erori permite această limită unilaterală de încredere de 95% în modelul declarat, nu concluzia că eroarea este imposibilă. Pentru n mare, aproximația 3/n este utilă deoarece minus logaritmul natural al lui 0,05 este aproape trei.

Interpretarea frecventistă privește acoperirea procedurii pe repetări ale experimentului, nu o probabilitate de 95% atribuită parametrului fix fără un model bayesian. Limita nu se aplică nemodificat când candidatul a fost ales după încercarea multor variante pe aceleași teste.

A8. Un control simplu pentru o familie finită de candidați

Avem K candidați fixați înainte de evaluare, cu K întreg pozitiv, iar e și d sunt numere strict între zero și unu. Pentru orice candidat cu eroare reală de cel puțin e, probabilitatea să treacă fără eroare n teste independente este cel mult (1 − e) la puterea n, care este cel mult exp(−n × e).

Probabilitatea ca măcar unul dintre cei K candidați răi să treacă este cel mult suma acestor probabilități: K × exp(−n × e). Nu este necesară independența dintre candidați pentru această inegalitate a reuniunii. Pentru ca limita să fie cel mult d, este suficient ca n să fie cel puțin [ln(K) + ln(1/d)] / e.

Aceasta este o garanție limitată: clasa trebuie fixată independent de setul de evaluare, testele trebuie să respecte modelul și criteriul este trecerea fără erori. O căutare adaptivă care modifică nelimitat clasa după rezultate cere o altă analiză. Formula explică de ce numărul încercărilor contează, nu rezolvă singură întreaga validare MRP.

ANEXA B

Exerciții cu soluții comentate

B1. Mulțime sau secvență?

Problema: un agent stochează numai mulțimea operațiilor „adaugă 1” și „înmulțește cu 2”. Pornind de la zero, poate determina rezultatul oricărei secvențe de două operații în care fiecare apare o dată?

Soluție: nu. Adunarea urmată de înmulțire produce doi; înmulțirea urmată de adunare produce unu. Mulțimea este aceeași, rezultatul diferă. O reprezentare suficientă trebuie să păstreze ordinea sau o altă informație din care rezultatul se poate calcula. Numărul de operații nu repară coliziunea, fiindcă este doi în ambele cazuri.

B2. Ce trebuie rafinat?

Problema: jurnalul păstrează ordinea corectă, dar interpretorul alege primul eveniment. Este necesară o nouă reprezentare?

Soluție: nu pentru această cauză. Informația necesară există. Trebuie corectată operația de selecție. Un sistem care adaugă o reprezentare nouă poate repara accidental rezultatul, dar ar rata diagnosticul și ar introduce complexitate inutilă.

B3. Zgomot sau coliziune?

Problema: aceeași reprezentare produce uneori succes și alteori eșec. Este demonstrată insuficiența ei?

Soluție: nu. Dacă procesul este stocastic, ambele rezultate pot proveni din aceeași distribuție. Sunt necesare repetări sau un model care să arate că două situații cu aceeași reprezentare induc distribuții diferite. Un singur dezacord între etichete nu distinge variabilitatea de informația pierdută.

B4. Relația lipsă

Problema: x și y au fiecare valori între zero și zece, iar agentul trebuie să demonstreze că x − y este zero. Ce informație suplimentară este suficientă?

Soluție: relația x = y este suficientă. Micșorarea separată a intervalelor poate ajuta în cazuri particulare, dar nu surprinde în general dependența. Exemplul arată de ce o reprezentare relațională poate fi mai importantă decât o precizie numerică mai mare a fiecărei variabile luate separat.

B5. Când merită învățarea?

Problema: construirea și verificarea unui interpretor costă 1.000 ms. Costul vechi este 40 ms pe sarcină, iar cel nou, inclusiv selecția, este 10 ms. Câte utilizări sunt necesare pentru un câștig strict?

Soluție: economisirea este 30 ms pe utilizare. Sunt necesare mai mult de 1.000/30 utilizări, deci 34. La 34, costul vechi este 1.360 ms, iar cel nou 1.340 ms. Rezultatul presupune că acuratețea rămâne comparabilă și că nu apar costuri suplimentare de invalidare.

B6. O vecinătate care nu este echivalență

Problema: trei predicții binare au probabilitățile 0,00, 0,06 și 0,12. La pragul 0,08, putem reuni toate predicțiile apropiate într-o clasă cu diametru cel mult 0,08?

Soluție: nu. Prima este apropiată de a doua și a doua de a treia, dar prima și a treia sunt la distanța 0,12. Închiderea tranzitivă a relației de apropiere poate depăși pragul. Algoritmul de grupare trebuie să controleze diametrul sau să accepte explicit o altă garanție.

B7. Conul unei proceduri

Problema: două proceduri au utilitățile nete (0,8; 0,2) și (0,4; 0,6) în două stări. Pentru ce probabilitate p a primei stări este prima preferabilă?

Soluție: prima are valoarea 0,2 + 0,6p, a doua 0,6 − 0,2p. Comparația dă 0,8p ≥ 0,4, deci p ≥ 0,5. În ponderi nenormalizate, condiția este z1 ≥ z2. Aceasta definește un con în cadranul nenegativ, cu egalitate de preferință pe frontieră.

B8. O compresie geometrică

Problema: reprezentăm un punct prin $x^2 + y^2$. Putem răspunde atât dacă se află într-un inel concentric, cât și dacă se află la stânga originii?

Soluție: prima întrebare poate fi rezolvată prin compararea razei pătratice cu pragurile. A doua nu: punctele (1; 0) și (−1; 0) au aceeași reprezentare. Simplificarea geometriei a fost obținută prin pierderea direcției, nu prin conservarea tuturor informațiilor.

B9. Dovadă corectă, concluzie nesusținută

Problema: un solver dovedește că un rezumat respectă regulile extrase dintr-un document. Putem declara că rezumatul este fidel documentului?

Soluție: numai dacă este justificată și corectitudinea extragerii regulilor relevante. Verificarea internă nu acoperă automat traducerea din text. Trebuie păstrate două evidențe distincte: legătura dintre document și reprezentare, apoi legătura dintre reprezentare și rezumat.

B10. Cum se pierde o ipoteză favorită

Problema: sistemul MRP depășește un model simplu, dar nu depășește o bibliotecă fixă cu selector, după includerea tuturor costurilor. Ce concluzie este defensibilă?

Soluție: rezultatul susține utilitatea specializării sau a selecției în experiment, nu avantajul inducției adaptive. Trebuie investigat dacă biblioteca fixă acoperă deja toate distincțiile necesare ori dacă inducția este prea costisitoare. Nu este legitim să atribuim sistemului complet un avantaj pe care componenta sa distinctivă nu l-a produs.

ANEXA C

Glosar operațional

Abstracție. Reprezentare care păstrează anumite proprietăți și ignoră altele. Valoarea sa depinde de întrebările pentru care este folosită.

Abținere. Rezultat în care agentul nu formulează răspunsul cerut, deoarece informația, verificarea sau bugetul sunt insuficiente.

Adaptor. Procedură care transformă o reprezentare în alta, cu un contract privind informația păstrată și pierdută.

Agent. Sistem care primește observații, menține o stare și alege operații sau acțiuni în raport cu sarcini și constrângeri.

Atlas de interpretoare. Organizare propusă a mai multor reprezentări locale și a adaptoarelor dintre ele; nu presupune o varietate geometrică.

Bibliotecă adaptivă. Colecție versionată de interpretoare care poate fi extinsă, consolidată, suspendată și evaluată prin reutilizare.

Calibrare. Concordanță între probabilitățile declarate și frecvențele relevante ale rezultatelor; nu este identică cu acuratețea.

Circuit. Compoziție executabilă reprezentată prin operații și dependențe între valorile lor.

Coliziune relevantă. Identificarea prin aceeași reprezentare a unor situații care cer răspunsuri sau distribuții diferite pentru același test.

Con convex. Mulțime închisă la combinații liniare cu coeficienți nenegativi; în carte poate descrie regiuni de preferință între proceduri.

Consolidare. Promovarea și reorganizarea unor proceduri pentru reutilizare, sub criterii explicite de evaluare și cost.

Contract. Specificație a intrărilor, ieșirilor, precondițiilor, efectelor și limitelor unei operații.

Contraexemplu. Caz care infirmă o afirmație precisă. În CEGAR trebuie verificat dacă exemplul abstract corespunde unui comportament concret.

Distincție executabilă. Diferență relevantă pe care un program o poate calcula din informațiile disponibile și care poate fi evaluată.

Echivalență comportamentală. Indistinguibilitate în raport cu o familie declarată de continuări sau teste.

Episod. Înregistrare contextualizată a observațiilor și acțiunilor, cu identitățile și ordinea relevante.

Epocă de execuție. Interval logic în care o sarcină folosește versiuni stabile ale interpretoarelor și politicii.

Evidență. Artefact care susține un statut delimitat: observație, test, derivare, demonstrație sau evaluare independentă.

Factorizare. Posibilitatea de a obține un răspuns corect printr-o reprezentare intermediară și o funcție aplicată acesteia.

Generalizare. Performanță pe cazuri care nu au fost folosite pentru construirea sau selectarea soluției, în distribuții explicit descrise.

Inducție de programe. Construirea unor programe candidate din exemple, specificații și un vocabular de operații.

Interpretor. Procedură care transformă intrări și stare în rezultate, stare nouă și evidențe, sub condiții declarate.

Interpretare abstractă. Familie de metode pentru aproximarea corectă a comportamentului programelor în domenii care păstrează proprietăți selectate.

Intervenție. Acțiune care modifică o componentă a sistemului studiat; efectul ei nu este identic, în general, cu o condiționare observațională.

Invariant. Proprietate care se păstrează prin tranzițiile relevante ale unui sistem, sub ipoteze explicite.

Meta-controler. Componentă care selectează operații de calcul, observație, inducție sau oprire, ținând seama de costuri.

MRP. Programul de cercetare al construirii, selectării și revizuirii unor interpretoare prin consecințe verificabile și constrângeri de resurse.

MRP-VM. Posibil mediu de execuție care administrează starea, dependențele, interpretoarele și condițiile de acceptare ale operațiilor MRP.

Politică. Regulă care alege acțiuni în funcție de informația disponibilă; poate include reacții la rezultate intermediare.

Proveniență. Informație despre sursele și transformările unui rezultat; facilitează auditul, dar nu garantează adevărul.

Pseudometrică. Funcție de distanță care respectă nenegativitatea, simetria și inegalitatea triunghiului, dar poate fi zero între obiecte distincte.

Rafinare. Adăugarea unor distincții fără pierderea celor păstrate anterior, atunci când termenul este folosit în sensul strict al partițiilor.

Reprezentare predictivă. Descriere a stării prin predicții despre rezultatele unor teste, în locul identificării obligatorii a unei stări latente.

SOP Lang. Opțiune de limbaj pentru compoziții executabile prin circuite, discutată aici ca posibil suport al MRP, nu ca premisă a teoriei.

Suficiență. Păstrarea informației necesare unei familii declarate de răspunsuri sau predicții; nu înseamnă păstrarea tuturor informațiilor.

Transfer. Reutilizarea unei capacități în alte situații sau familii de sarcini, cu verificarea condițiilor de aplicare.

Valoarea calculului. Câștigul decizional așteptat produs de o operație internă, după scăderea costurilor relevante.

Verificare formală. Stabilirea unei proprietăți pentru toate cazurile acoperite de un model și de ipotezele demonstrației, nu pentru orice lume posibilă.

ANEXA D

Bibliografie comentată

Sursele de mai jos fundamentează conceptele utilizate, nu validează arhitectura MRP ca întreg. Identificatorii corespund citărilor din text. Anul publicației originale este separat de anul depunerii în arXiv acolo unde diferă. Adresele au fost consultate pentru această ediție în septembrie 2026. Selecția include lucrări originale, expuneri ale autorilor și documentație instituțională.

[Angluin1987] Învățarea prin interogări

Angluin, Dana (1987). Learning regular sets from queries and counterexamples. Information and Computation, 75(2), 87–106. DOI: 10.1016/0890-5401(87)90052-6.

Lucrare fundamentală pentru învățarea limbajelor regulate prin interogări și contraexemple. În carte este folosită ca antecedent al distincției dintre un model învățat și un test care îl poate separa de sistemul țintă; nu ca garanție pentru orice mediu MRP.

Înregistrarea publicației

[Barak] Computabilitate și complexitate

Barak, Boaz. Introduction to Theoretical Computer Science. Manual online al autorului, versiune consultată în septembrie 2026.

Referință pentru limitele calculabilității și diferența dintre existența matematică a unei soluții și calcularea ei eficientă. Aceste limite motivează limbajele restrânse, bugetele și rezultatele neconcludente din arhitectura propusă.

Manualul autorului

[Boyd2004] Convexitate

Boyd, Stephen; Vandenberghe, Lieven (2004). Convex Optimization. Cambridge University Press.

Fundament pentru mulțimi convexe, conuri, semispații și optimizare. Derivarea regiunilor de preferință din carte este un calcul didactic sub ipoteze explicite despre proceduri și utilități, nu un rezultat empiric despre geometria cunoașterii.

Pagina cărții și textul pus la dispoziție de autori

[Callaway2018] Alegerea calculelor

Callaway, Frederick; Gul, Sayan; Krueger, Paul M.; Griffiths, Thomas L.; Lieder, Falk (2018). Learning to select computations. Proceedings of the Conference on Uncertainty in Artificial Intelligence. Preprint inițial: arXiv:1711.06892, 2017.

Referință directă pentru învățarea unor politici aproximative de metaraționament. Susține poziționarea valorii calculului ca fundament existent, distinct de ipoteza MRP privind construirea și consolidarea unor noi interpretoare.

Manuscrisul autorilor

[Clarke2000] Rafinarea ghidată de contraexemple

Clarke, Edmund; Grumberg, Orna; Jha, Somesh; Lu, Yuan; Veith, Helmut (2000). Counterexample-Guided Abstraction Refinement. Computer Aided Verification, Lecture Notes in Computer Science, 1855, 154–169. DOI: 10.1007/10722167_15.

Sursa pentru bucla CEGAR. Cartea distinge contraexemplul abstract artificial de o eroare predictivă obișnuită și propune o integrare inspirată de această disciplină, fără transferul automat al garanțiilor.

Publicația originală

[Cousot1977] Interpretarea abstractă

Cousot, Patrick; Cousot, Radhia (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of the ACM Symposium on Principles of Programming Languages, 238–252.

Fundamentul interpretării abstracte. În argumentul MRP, arată că pluralitatea domeniilor de interpretare poate avea obligații de corectitudine precise și că aproximarea nu este sinonimă cu euristica neverificată.

Pagina publicației la autori

[CousotIntro] Precizie, aproximare și puncte fixe

Cousot, Patrick. Abstract Interpretation in a Nutshell. Expunere introductivă online a autorului.

Lectură complementară pentru relația dintre analiza concretă, domeniile abstracte și mecanismele de stabilizare. Este recomandată înaintea aprofundării detaliilor matematice pe care cartea le introduce numai în măsura necesară arhitecturii.

Expunerea autorului

[DeSabbata2025] Metaraționament pentru LLM-uri

De Sabbata, C. Nicolò; Sumers, Theodore R.; AlKhamissi, Badr; Bosselut, Antoine; Griffiths, Thomas L. (2025). Rational Metareasoning for Large Language Models. arXiv:2410.05563, versiunea 3, 23 iunie 2025; prima versiune depusă în 2024.

Preprint relevant pentru poziționarea selecției raționale a calculului în sisteme cu modele lingvistice. Cartea nu revendică drept noutate MRP simpla alocare adaptivă a efortului de raționare.

Versiunea citată

[Dwork2015] Evaluare adaptivă

Dwork, Cynthia; Feldman, Vitaly; Hardt, Moritz; Pitassi, Toniann; Reingold, Omer; Roth, Aaron (2015). Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629.

Referință pentru riscurile reutilizării adaptive a datelor de evaluare. Motivează separarea dintre inducție, dezvoltare și evaluarea finală; nu este folosită pentru a afirma că un singur procedeu rezolvă toate riscurile statistice ale MRP.

Manuscrisul autorilor

[Ellis2020] Biblioteci de programe

Ellis, Kevin; Wong, Catherine; Nye, Maxwell; Sable-Meyer, Mathias; Cary, Luc; Morales, Lucas; Hewitt, Luke; Solar-Lezama, Armando; Tenenbaum, Joshua B. (2020). DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. arXiv:2006.08381.

Lucrare care leagă învățarea de abstracții programatice de ghidarea căutării. Este un precedent concret pentru biblioteca de interpretoare; MRP trebuie să își justifice separat mecanismul de diagnostic, selecție și verificare a distincțiilor.

Manuscrisul cu titlul citat

[Ferns2004] Distanțe comportamentale

Ferns, Norman; Panangaden, Prakash; Precup, Doina (2004). Metrics for Finite Markov Decision Processes. Proceedings of the Conference on Uncertainty in Artificial Intelligence, 162–169. Copie în arXiv:1207.4114, depusă în 2012.

Antecedent pentru metrici comportamentale în procese decizionale. Pseudometrica definită în carte prin familia testelor nu este identificată automat cu metrica exactă din această lucrare.

Copia autorilor

[Ganea2018] Conuri de implicare

Ganea, Octavian; Bécigneul, Gary; Hofmann, Thomas (2018). Hyperbolic Entailment Cones for Learning Hierarchical Embeddings. Proceedings of the 35th International Conference on Machine Learning, PMLR 80, 1646–1655.

Sursă pentru reprezentarea unor relații ierarhice prin conuri. Cartea separă această motivație, bazată pe ordine parțială, de derivarea conurilor de preferință din utilități așteptate.

Publicația și materialele autorilor

[Gardenfors2000] Spații conceptuale

Gärdenfors, Peter (2000). Conceptual Spaces: The Geometry of Thought. MIT Press. Pagina editorială indicată include ediția paperback din 2004.

Reper pentru reprezentarea conceptelor prin dimensiuni de calitate și regiuni geometrice. În MRP, forma unei regiuni este tratată ca ipoteză dependentă de coordonate, operații și sarcini.

Pagina editurii

[Gneiting2007] Evaluarea probabilităților

Gneiting, Tilmann; Raftery, Adrian E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association, 102(477), 359–378. DOI: 10.1198/016214506000001437.

Fundament pentru scoruri de evaluare a predicțiilor probabiliste. Distincția dintre probabilitate, calibrare și utilitate este necesară pentru a nu confunda performanța unui agent cu sinceritatea sau adevărul estimărilor sale.

Textul pus la dispoziție de autori

[Grunwald2004] Lungimea descrierii

Grünwald, Peter (2004). A tutorial introduction to the minimum description length principle. arXiv:math/0406077.

Introducere în MDL. Susține utilizarea complexității descrierii ca regularizare a programelor candidate, cu precizarea că simplitatea depinde de limbaj și nu garantează singură adevărul sau transferul.

Tutorialul autorului

[Harnad1990] Ancorarea simbolurilor

Harnad, Stevan (1990). The Symbol Grounding Problem. Physica D: Nonlinear Phenomena, 42, 335–346. DOI: 10.1016/0167-2789(90)90087-6. Copie în arXiv:cs/9906002, depusă în 1999.

Reper filosofic pentru legătura dintre simboluri și ceea ce ele reprezintă. Cartea propune ancorări operaționale prin surse, observații și intervenții, fără a pretinde rezolvarea integrală a problemei.

Copia autorului

[Jordan1993] Specializare și rutare

Jordan, Michael I.; Jacobs, Robert A. (1993). Hierarchical mixtures of experts and the EM algorithm. Proceedings of the International Joint Conference on Neural Networks, 1339–1344.

Precedent pentru o arhitectură cu experți și mecanisme ierarhice de ponderare. Este citată versiunea de conferință din 1993 disponibilă la adresa de mai jos, nu articolul extins cu titlu similar din 1994.

Textul lucrării de conferință

[Pearl2018] Observație și intervenție

Pearl, Judea (2018). Theoretical Impediments to Machine Learning With Seven Sparks from the Causal Revolution. arXiv:1801.04016.

Referință pentru diferența dintre relații observaționale, intervenții și întrebări cauzale. Exemplele calculate în carte sunt independente și restrânse; nu transformă reprezentarea predictivă într-o teorie cauzală universală.

Manuscrisul autorului

[Shannon1948] Informație și codificare

Shannon, Claude E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379–423 și 623–656.

Fundament istoric pentru tratarea cantitativă a informației. Cartea folosește distincția dintre suportul codificat și informația pierdută, fără a echivala cantitatea de informație cu sensul sau înțelegerea.

Copie academică a textului original

[Singh2004] Reprezentări predictive ale stării

Singh, Satinder; James, Michael R.; Rudary, Matthew R. (2004). Predictive State Representations: A New Theory for Modeling Dynamical Systems. Proceedings of the Conference on Uncertainty in Artificial Intelligence, 512–519. Copie în arXiv:1207.4167, depusă în 2012.

Sursă centrală pentru starea descrisă prin predicții asupra testelor. Cartea preia această perspectivă ca fundament al echivalenței relevante, fără a presupune identificarea perfectă a distribuțiilor din date finite.

Copia autorilor

[SoftwareFoundations] Contracte și corectitudine

Software Foundations, volumul 2, Programming Language Foundations. Capitolul Hoare: Hoare Logic, Part I. Manual colectiv online, versiune consultată în septembrie 2026.

Referință didactică pentru precondiții, postcondiții și corectitudine parțială. În MRP, un contract verificat privește modelul și ipotezele sale; nu confirmă automat adevărul intrărilor externe.

Capitolul din manual

[Tappler2019] Învățarea sistemelor reactive

Tappler, Martin; Aichernig, Bernhard K.; Bacci, Giovanni; Eichlseder, Maria; Larsen, Kim G. (2019). L*-Based Learning of Markov Decision Processes (Extended Version). arXiv:1906.12239.

Exemplu de extindere a învățării active către procese decizionale din clasa studiată de autori. Este relevantă separarea dintre condițiile ideale ale interogărilor și aproximarea lor prin testare practică.

Versiunea extinsă

[W3C2013] Proveniență interoperabilă

Moreau, Luc; Missier, Paolo, editori (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 aprilie 2013.

Model pentru descrierea entităților, activităților, agenților și derivărilor. Poate inspira registrul de proveniență MRP. Înregistrarea derivării permite auditul, dar nu certifică adevărul rezultatului derivat.

Recomandarea W3C

[Willsey2021] Rescriere și echivalență

Willsey, Max; Nandi, Chandrakana; Wang, Yisu Remy; Flatt, Oliver; Tatlock, Zachary; Panchekha, Pavel (2021). egg: Fast and Extensible Equality Saturation. Proceedings of the ACM on Programming Languages, 5, POPL. DOI: 10.1145/3434304. Preprint: arXiv:2004.03082.

Referință pentru organizarea și explorarea expresiilor echivalente prin saturare. Într-un MRP, utilizarea sa cere reguli de rescriere justificate; nu stabilește echivalența arbitrară a programelor generate.

Manuscrisul autorilor